

MATLAB

MATLAB bir yorumlayıcıdır; yani sonuç daha ziyade el tipi hesap makinelerine benzer tarzda ekranda yazılı metni olarak alınabilir. sonuçta diğer dillerde olduğu gibi " derleme" ' ye gerek duyulmaz ama programlamaya da izin vermesi yönüyle çok güçlü bir paket programdır.

* 1970'lerin sonunda Clever Moler tarafından yazılan Matlab programının tipik kullanım alanları :

- 1-) Matematiksel hesaplama işlemleri
- 2-) Algoritma geliştirme ve kod yazma
- 3-) Lineer cebir, istatistik, fourier analizi, filtreleme, optimizasyon, sayısal integrasyon v.b. konularda matematik fonksiyonlar
- 4-) 2D ve 3D grafiklerinin çizimi
- 5-) Modelleme ve simülasyon (benzetim)
- 6-) Grafikselleme arayüz oluşturma
- 7-) Veri analizi ve kontrolü
- 8-) Gerçek dünya şartlarında uygulama geliştirme şeklinde özetlenebilir.

MATLAB KOMUTLARINDAN BAZILARI

Çalışma Ortamı Komutları

- help**- Fonksiyon yardım komutu
- demo,intro** -Demo ve tanıtım komutu
- who**- Kayıtlı değişkenleri görüntüleme
- whos**- Kayıtlı değişkenlerin türlerini görüntüleme
- pwd, cd**-Klasör gösterme, değiştirme komutu
- size**- Matris boyut komutu
- length**- Matris uzunluk komutu
- tic, toc**- Zaman başlatma ve görüntüleme komutu
- quit,exit**- MATLAB'dan çıkış komutu
- clear**- Çalışma ortamındaki tüm değişkenleri silme komutu
- clear x**- Çalışma ortamındaki x değişkenini silme komutu
- save**- çalışma ortamını matlab.mat olarak kaydetme butonu
- save ad**- çalışma ortamını ad.mat olarak kaydetme butonu
- save ad x y**- çalışma ortamını ad.mat olarak kaydetme butonu
- load ad**- Çalışma ortamına ad.mat' ı yükleme komutu
- print**- Çalışma alanını veya şekli yazdırma komutu

Matematiksel Operatörler

- | | |
|---|------------|
| + | - Toplama |
| - | - Çıkarma |
| * | - Çarpma |
| / | - Bölme |
| ^ | - Üst alma |

<code>.*</code>	- Elemanter çarpma
<code>./</code>	- Elemanter Bölme
<code>.</code>	- Elemanter üst alma
<code>sqrt</code>	- Karekök alma,

Mantıksal Operatörler

<code>></code>	- Büyüktür ifadesi
<code>>=</code>	- Büyük eşittir ifadesi
<code><</code>	- Küçüktür ifadesi
<code><=</code>	- Küçük eşittir ifadesi
<code>==</code>	- Döngülerde eşittir ifadesi
<code>~=</code>	- Eşit değildir ifadesi
<code>& ~</code>	- Ve, veya, değil ifadeleri

Sabit ifadeler

<code>pi</code>	- π
<code>inf</code>	- Sonsuz
<code>nan</code>	- Sayı değil
<code>i, j</code>	- imajiner i ve j
<code>eps</code>	- 2.2204e-016 değeri

Trigonometrik Fonksiyonlar

<code>cos, sin, tan, cot</code>	- Trigonometrik ifadeler
<code>acos, asin, atan, acot</code>	- Ters trigonometrik ifadeler
<code>cosh, sinh, tanh, coth</code>	- Hiperbolik ifadeler
<code>acosh, asinh, atanh, acoth</code>	- Ters hiperbolik ifadeler

Logaritma Fonksiyonları

<code>exp</code>	- eksponansiyel ifadesi
<code>log</code>	- ln anlamlı logaritma ifadeleri
<code>log10</code>	- 10 tabanı'nda logaritma
<code>log2</code>	- 2 tabanı'nda logaritma
<code>pow2</code>	- 2 tabanı'nda kuvvet

Kompleks Fonksiyonlar

<code>abs</code>	-Mutlak deger
<code>angle</code>	-Radyan olarak faz açısı

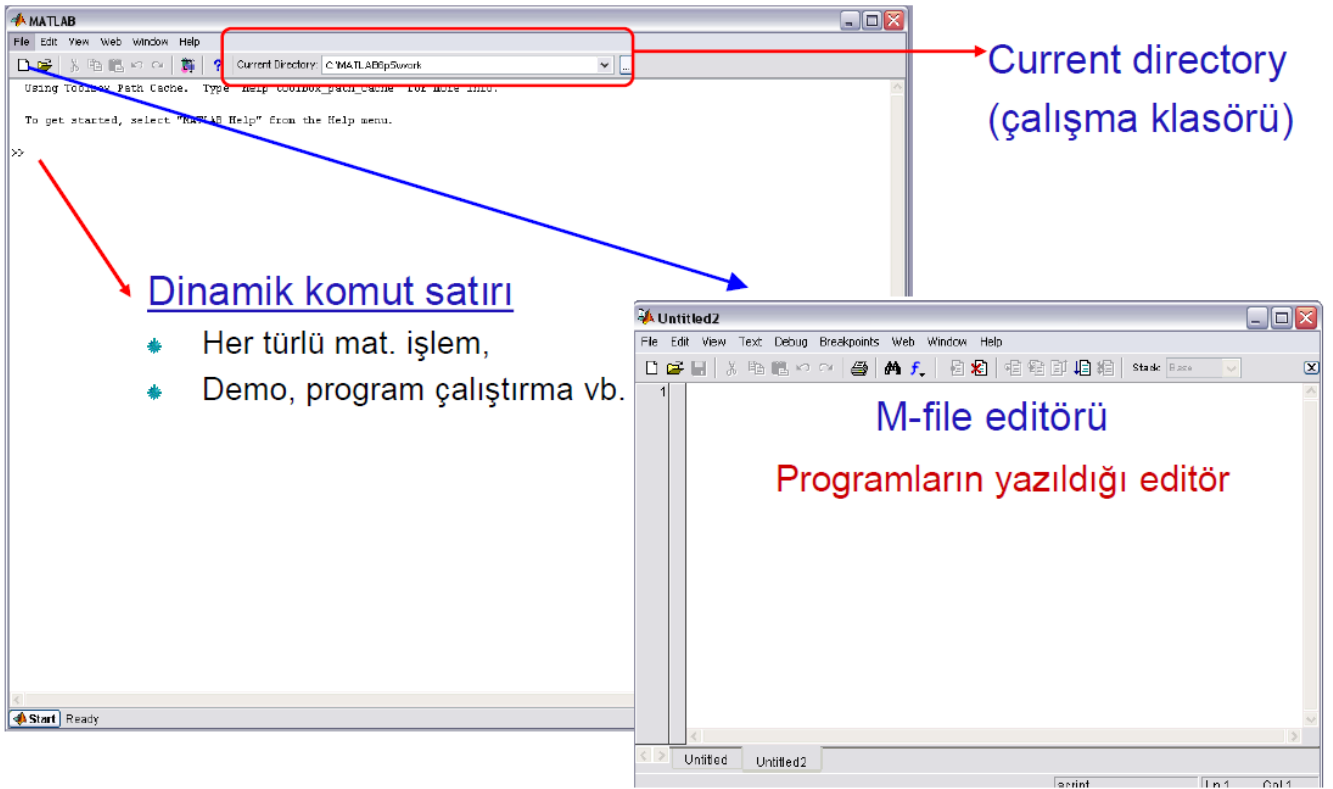
conj	-Kompleks eşlenik
imag	-imajiner Kısım
real	- Reel kısım
complex	- Reel ve imajiner kısımlarla kompleks sayı

Yuvarlama Fonksiyonları

fix	-Sıfıra yuvarlama komutu
floor	-Negatif sonsuza yuvarlama komutu
ceil	-Pozitif sonsuza yuvarlama komutu
round	-En yakın tam sayıya yuvarlama komutu
rem	-Bolme işleminden sonra kalan ifadesi
sign	-Signum fonksiyonu

Matematiksel işlemler

sum, cumsum	- Toplama ve kümülatif toplama
diff	- Çıkarma
mean	- Ortalama ifadesi
median	-Orta değeri gösterme komutu
min, max	-Minimum ve maksimum ifadeleri
sort	- Matris elemanlarını küçükten büyüğe sıralama
prod	- Seri çarpım
factorial	- Faktöryel alma
primes	-Asal sayıları listele
gcd	-En büyük ortak bölen
lcm	-En küçük ortak bölen
rat	-Rasyonel tahmin
rats	-Rasyonel çıkış
perms	-Bütün olası kombinasyonları



Kırmızı=başlık ve simgeler ; Yeşil= açıklama satırları ; Mavi=komutlar

* Özel bir komut hakkında bilgi edinmek için komut satırına **"help komut adı"** komutu girilir.

(Burada **"komut adı"** ilgilenilen MATLAB komut kelimesidir.)

* Yukarı aşağı ok yönleri önceki işlemleri gösterir.

- * **>> chdir** : Çalışılan dizini gösterir.
- * **>> dir** : Bulunulan dizindeki bütün dosyaları görüntüler.
- * **>> cd (dizin ismi)** : İstenen dizine geçiş sağlar.
- * **>> what** : MATLAB dosyalarını ayrı ayrı gösterir.
- * **>> del(dosya ismi)** : Dosyayı siler.
- * **>> type(dosya ismi)** : Dosyanın içeriğini ekrana getirir.

* **undefined function or variable 'f'** : Bu fonksiyona daha önce bir değişken atamamışsın.

* Kullanıcı , belirli bir konuda hangi komutları kullanabileceğini bilmek isterse **"lookfor kavram adı"** komutunu girmesi yeterlidir.

(ÖRN: **"lookfor exponential"** yazılıp enter tuşuna basıldığında ; MATLAB her biri için kısa açıklamalarıyla birlikte üs almaya ilişkin tüm komutları sıralar.)

SÜTUN : **SATIR :**

* Değişken isimleri küçük büyük harf kullanımına duyarlıdır. Buna göre aynı anlama gelen fakat farklı yazılan **"orta"**, **"Orta"**, **"orTa"**, ve **"ORTA"** kelimeleri MATLAB için farklı değişkenlerdir.

* Değişken adları en fazla 31 karakter içerebilir. Bundan fazla olanlar dikkate alınmaz.

* Değişken isimleri daima bir harf ile başlamalı ve bunu herhangi bir sayıda harfler , rakamlar veya alt çizgi **"_"** izleyebilir. Noktalama işaretleri değişken isminde kullanılmaz. Çünkü bunların pek çoğunun MATLAB için bir anlamı vardır.

- * **clc** :ekran temizler.
- * **clear all** : herşeyi temizler.
- * **clear b** : Yalnızca "b" değişkenini siler.
- * **demo** : MATLAB demosunu çalıştırır.
- * **Date** : gün-ay-yıl görüntüler.
- * **Exit** : MATLAB oturumundan çıkar.
- * **who** : Kullanıcı tarafından tanımlanan değişkenlerin isimlerini listeler.
- * **whos** : Kullanıcı tarafından tanımlanan değişkenlerin isimlerini ve boyutlarını listeler.
- * **% (Yüzde)** : Açıklama satırları için kullanılır. % işaretinden sonraki ifadeler yürütülemez.
 ÖRN : >> k = 0.55 % kesirli ifadeyi belirtmek için
- * **= (Eşittir)** : İşaretin sağındaki ifadeyi solundaki değişkene atamak için kullanılır.
 ÖRN : >> m = - 12 % m değişkenine -12 değerini atar.
- * **[] (Köşeli parantez)** : Vektör ve matrisleri biçimlendirmek için kullanılır. Herhangi bir A matrisi komut satırında aşağıdaki gibi tanımlanabilir.
 ÖRN : >> A = [1 2 3 ; -2 1 0.5]
- * **;(Noktalı virgül)** :
 a-) Bir ifadenin yürütülmemesiyle elde edilen sonucun ekranda görünmemesi için kullanılır.
 ÖRN : >> a = 0.55 , b = 12 , c = -155;
 b-) Satırın sonlandırılması için kullanılır.
 ÖRN : >> A = [1 2 3 ; 4 5 6]
 c-) Bir satırın sonlandırılıp sonraki satırın oluşturulması için kullanılır.
 ÖRN : >> [1 2 3 ; 4 5 6 ; 7 8 9]
 ans =
 1 2 3
 4 5 6
 7 8 9
- * **() (Normal parantez)** :
 a-) Aritmetik ifadelerde işlem önceliğini belirtmek için kullanılır.
 ÖRN : >> k = x + 2 * (y + z)
 b-) Fonksiyon parametrelerini belirtmek ve kapatmak için veya dizi elemanlarını belirtmek için kullanılır.
 ÖRN : >> y = sin (x)
- * **.(Nokta)** :
 a-) Kesirli sayıları belirtmek için kullanılır.

ÖRN : >> k = 0.55

b-) A ve B gibi dizileri eleman elemana çarpmak için kullanılır.

ÖRN : >> A = [1 2 3 4 ; 5 6 7 8] , B = [1 2 3 4 ; 5 6 7 8]

A =

1 2 3 4
5 6 7 8

B =

1 2 3 4
5 6 7 8

>> A .* B

ans =

1 4 9 16 % (1 * 1) (2 * 2) (3 * 3) (4 * 4)
25 36 49 64 % (5 * 5) (6 * 6) (7 * 7) (8 * 8)

*** ... (Üç nokta) :** Tek bir satıra sığmayan ifadelerin alttaki satırdan devam ettiğini göstermek için kullanılır.

ÖRN : >> x = 1 + 2 + 3 + 4 + 5 + 6 + ... % ifade bu satıra sığmadığı için alttaki satırdan devam ediyor.
10 + 11 + 12

***, (Virgül) :**

a-) Bir satırdaki üç ayrı bildirimin gösterilmesi için kullanılır.

ÖRN : >> k = 0.55 , y = 3 + k , z = y ^ 2

b-) A Satır vektörünün elemanlarını ayırmak için kullanılır.

ÖRN : >> A = [4 , -3 , 5 , 0.833]

c-) Fonksiyonun parametrelerinin ayrılması için kullanılır.

ÖRN : >> B = conv (y , z)

*** ' (Tırnak) :**

a-) Text ifadelerinin aktarımında kullanılır.

ÖRN : >> ' Bir sayı giriniz : '

b-) Matrislerin evriğinin (Transpoz'unun) alınmasında kullanılır.

ÖRN : >> A = [1 2 3 ; 4 5 6 ; 7 8 9] ; B = A'

B =

1 4 7
2 5 8
3 6 9

*** : (İki nokta üst üste) :**

a-) Sütun işareti için kullanılır.

ÖRN : >> A(:, 2) % İkinci sütundaki bütün satırlar

ans =

2
5
8

b-) Satır işareti için kullanılır.

ÖRN: >> A(2,:) % İkinci satırdaki bütün sütunlar

ans =
4 5 6

c-) Düzenli artış oluşturmak için kullanılır.

ÖRN: >> x = 1 : 15 : 150

x =
1 16 31 46 61 76 91 106 121 136

veya

>> for k = 0 : 2 : 10

d-) İlk iki satırı görmek istiyorsak;

>> a(1:2,:)

ans =
1 2 3
4 5 6

Komut satırına (>>) bir ifade yazılıp giriş (enter) tuşuna basıldığında MATLAB bu komutu yürütür ve sonucu komut penceresinde gösterir.

SİMGE	İŞLEM	AÇIKLAMA
+	$a + b$	Toplama
-	$a - b$	Çıkarma
*	$a * b$	Çarpma
/	a / b	Sağa bölme
\	$a \backslash b$	Sola bölme
^	$a ^ b$	Üs alma
=	$a = b$	Atama
.*	$a .* b$	Eleman elemana çarpma
./	$a ./ b$	Eleman elemana sağa bölme
.\	$a .\ b$	Eleman elemana sola bölme
.^	$a .^ b$	Eleman elemana üs alma

ÖRN: >> A = [1 2 3], B = [4 5 6]

>> A ./ B

ans =
0.2500 0.4000 0.5000

>> A .\ B

ans =
4.0000 2.5000 2.0000

* >> e : 10 üssü demek

ÖRN:

3e9 = 3*10⁹

* MATLAB ile karşılaştırma işlemleri de yapılabilir.

Karşılaştırma işleminin sonucunda;

ifade doğru ise (1),

ifade yanlış ise (0),

sonuçları döndürülür.

Karşılaştırılan değişkenler dizi ise ; dizilerin boyutları eşit olmalı ve karşılaştırma işlemi eleman elemana olmalıdır.

SİMGE	İŞLEM	AÇIKLAMA
<	$a < b$	Küçük
>	$a > b$	Büyük
<=	$a \leq b$	Küçük eşit
>=	$a \geq b$	Büyük eşit
==	$a == b$	Eşittir
~=	$a \sim b$	Eşit değil

* MATLAB ' da mantıksal işlemlerde kullanmak mümkündür.

SİMGE	İŞLEM	AÇIKLAMA
~	$\sim b$	Değil (not)
&	$a \& b$	Ve (and)
	$a b$	Veya (or)

* MATLAB aynı zamanda pek çok hazır (built-in) matematik ve sayısal fonksiyonlara sahiptir.Bu fonksiyonların bazıları ;

MATLAB YAZILIŞI	FONKSİYON	AÇIKLAMA
sin(pi)	sin()	Sinüs
cos(pi)	cos()	Cosinüs
tan(pi)	tan()	Tanjant
asin(0)	arcsin()	Arksinüs
acos(0)	arccos()	Arkkosinüs
atan(0)	arctan()	Arktanjan
exp(2)	exp()	Ekspansiyon
log(10)	log()	Doğal logaritma
log10(10)	log10()	10 tabanlı logaritma
sqrt(25)	sqrt()	Kare kök
abs(3)	abs()	Mutlak değer

Not : Açık değerleri radyan olarak işlem görür.

ÖRN :

```
>> abs(-3)
ans =
3
```

ÖRN :

```
>> sqrt(85)
ans =
9.2195
```

ARİTMETİK İŞLEMLERDE ÖNCELİK SIRALAMASI :

* MATLAB' da yapılmak istenen işlem sonucunun doğru olabilmesi için , aritmetik işlem önceliğine dikkat edilmelidir.

Aritmetik işlemlerde öncelik sırası ;

- 1-) En içten en dışa paranteze doğru bütün parantez işlemleri
- 2-) Soldan sağa doğru bütün üstel işlemler
- 3-) Soldan sağa doğru bütün çarpma ve bölme işlemleri
- 4-) Soldan sağa doğru bütün toplama ve çıkarma işlemleri

ÖRN : >> x = 180 / 45 + 2 , y = sin(4 * x) - (2 * cos(x)) ^ 3


```
x=
6
y=
-7.9872
```

SAYILAR :

MATLAB 'da sayılar ;

- 1-) Pozitif (+) , negatif (-) , reel veya ondalık biçimde gösterilebilir.
- 2-) Ondalık sayılar , (Virgül) yerine . (Nokta) ile gösterilir.
- 3-) Bilimsel gösterimde "e" harfi 10' un kuvvetini temsil eder.
- 4-) Karmaşık sayılarda sana (İmajiner) kısımlar i veya j harfleriyle gösterilir.

* MATLAB 'da ondalık sayıları tam sayıya dönüştüren fonksiyonlar bulunmaktadır ;

ÖRN : >> a=3, c=-10.985, d=1.660541e-20, e= 3+2i % veya 3+2*i
>> x=3+ 2i veya 3+2*i veya 3+2j veya 3+2*j

* >> y=round (c) : c işaretine bakılmaksızın ; buçuğun altı bir alt tam sayıya , buçuk ve üzeri ise bir üst tam sayıya yuvarlanır.

ÖRN : >> y=round (c)
y=
-11
ÖRN: >> round(8.6)
ans=
9

* >> y=fix (c) : c değerinin işaretine bakılmaksızın sıfıra doğru yuvarlar.

ÖRN : >> y=fix (c)
y=
-11
ÖRN : >> fix(8.6)
ans=
8

* >> y=floor (c) : c değerinin işaretine bakılmaksızın $-\infty$ 'a doğru yuvarlar.

ÖRN : >> y=floor (c)
y=
-11
ÖRN: >> floor(-8.6)
ans =
-9

* >> y=ceil (c) : c değerinin işaretine bakılmaksızın $+\infty$ 'a doğru yuvarlanır.

ÖRN : >> y=ceil (c)

y=
-10

* >> y=sign (c) : c değerinin işaretini geri döndürür.

ÖRN : >> y=sign (c)

y=
-1

ÖRN: >> sign(-8)

ans =
-1

DEĞİŞKENLER :

* MATLAB yeni bir değişken adı tanımladığında , kendiliğinden değişkeni oluşturur ve uygun şekilde yerleştirir.Eğer aynı değişken daha önce tanımlanmış ise , değişkenin içeriğini değiştirir.

* Değişken adının tanımlanmasında dikkat edilecek hususlar ;

1-) Değişken adları kesinlikle ad ile başlar.

2-) Özel ve Türkçe karakterler değişken adında kullanılamaz. (* . , : / - ç ğ ... v.b.)

3-) Değişken adlarındaki büyük küçük harfler farklı değişken olarak algılanır.

4-) Değişken adlarında i ve j harflerinin kullanılmamasına özen gösterilmelidir.

ÖRN : >> a = 3 , b = 2 % Virgülle ayırarak aynı satırda birden çok atama yapılabilir.

a =

3

b =

2

>> c = a + b

c=

5

>> d = a ^ b

d=

9

* Bir değişkenin değerini görmek için değişkenin adını girmek yeterlidir

ÖRN : >> d

d=

9

* Değişkenlerin içeriği clear komutu ile silinebilir.

ÖRN : >> d=10

clear d % d değişkenleri siler.

clear all % bütün değişkenleri siler.

* MATLAB ' da bazı özel sayı veya değişkenler tanımlanmıştır

* >> ans : Belirtilmeyen değişkenler için

* >> pi : π sayısı için

* >> eps : En küçük sayı için

* >> NaN : Tanımsız işlem için

* >> inf : Sonsuz sayı için

* >> **i ve j** : Karmaşık sayıların sanal bileşenleri için kullanılan hazır değişkenlerdir.

ÖRN : >> **pi**
ans=
3.1416
>> **eps**
ans=
2.2204e-016

DEĞİŞKENLERE DEĞER GİRİŞİ :

* >> **input** : Bu komut ile değişkenlere dışarıdan değer girişi yapılabilmektedir.

Genel yazımı ;

* **değişken adı = input ('değişken : ')**
>> **y = input (' y sayısını giriniz : ')**
y sayısını giriniz :

DEĞİŞKENLERİ SAKLAMA VE TEKRAR GERİ ÇAĞIRMA :

* MATLAB' dan çıkıldığında tüm değişkenler silinir. Bu nedenle oturum boyunca çalışma alanında bulunan değişkenlere sonradan erişebilmek için bu değişkenler saklanması gerekir.

* >> **save** : Bu komut değişkenleri saklamak için kullanılır.

* >> **load** : Bu komut değişkenlere erişebilmek için kullanılır.

Kullanışı :

>> **save dosya adı**
>> **load dosya adı**
ÖRN : >> **a= 3 ; b= 2 ; c= 2-3j ;**
>> **save değişkenler**
>> **clear all**
>> **a**
??? undefined function or variable ' a '
>> **load değişkenler**
>> **a**
a=
3

VEKTÖR OLUŞTURMA :

MATLAB ' da vektörleri üretmek için kullanılabilen bir dizi yöntem vardır. Bunların bir kaçı burada sunulacaktır.

* Bir satır vektörü oluşturmanın yollarından biri ;

1-) Vektör elemanlarının boşluk ile ayrılması

ÖRN : >> **A= [1 2 3]**
A =
1 2 3

2-) Vektör elemanlarının virgül ile ayrılması

ÖRN : >> A = [1 , 2 , 3]

A =

1 2 3

* Bir sütun vektörü ve köşeli parantezin içerisinde ise ;

1-) Elemanların arasına noktalı virgül konularak
return)

2-) Her elemandan sonra giriş (enter veya

tuşuna basılarak da elde edilebilir.

ÖRN : >> B = [1 ; 2 ; 3]

1

2

3

ÖRN : >> B = [1

2

3]

B =

1

2

3

* Bir satır vektörün transpoz'unu almak, satır vektörünü sütun vektörüne dönüştürür. Aynı şekilde sütun vektörünün transpoz'uda , satır vektörüne dönüştürür.

* MATLAB 'da bir vektörün transpoz'unu almak için kesme (apostrof (')) işareti kullanılır.

ÖRN : >> A '

ans =

1

2

3

* Elemanları sabit bir artım ile düzenli bir vektör oluşturmak için (:) iki nokta üst üste kullanılabilir. Bu gösterim özellikle çok yüksek boyutlu ve sabit artışlı vektörlerin oluşturulmasında oldukça kullanılır.

Genel yazımı ;

X : Xmin : ΔX : Xmax

X : Atama yapılacak değişkenin adı

Xmin : Vektörün ilk elemanının değeri

ΔX : Artış miktarı

Xmax : Vektörün son elemanı

ÖRN : >> A = 1 : 1 : 6

% 1' den 6' ya kadar 1' er artımla 6 elemanlı satır vektörü oluşturur

A =

1 2 3 4 5 6

NOT : Artış pozitif olacağı gibi negatifte olabilir. Artış miktarı olarak " 1 " kullanılacaksa , artım değerinin verilmesi gerekmez ve bu durumda birer- birer artan sayılar üretilir.

ÖRN : >> B = 8 : -2 : 0

B =

8 6 4 2 0

>> A = 1 : 6

% Bu halde 1 artımı atlanmıştır.

A =

1 2 3 4 5 6

* Düzenli bir vektör oluşturmak için bir diğer yöntem linspace ve logspace komutlarıdır. Bu iki komutta iki nokta üst üste (:) gibi, iki sayı arasında düzenli artımlı birer satır vektörü oluşturur.

* **linspace** : Bu komut aralığı doğrusal olarak oluşturur.

* **logspace** : Bu komut aralığı logaritmik olarak oluşturur.

Genel kullanımı ;

$$\left. \begin{array}{l} \text{linspace} (X_{\min}, X_{\max}, n) \\ \text{logspace} (X_{\min}, X_{\max}, n) \end{array} \right\} \Delta x = \frac{X_{\max} - X_{\min}}{n-1}$$

Xmin : Vektörün ilk elemanının değeri

Xmax : Vektörün son elemanının değeri

n : İlk değer ile son değer arasındaki eleman sayısıdır. Eğer n parametresi belirtilmezse, iki nokta arası 100 eşit parçaya bölünür.

ÖRN : >> A = linspace (1 , 10 , 10)

A =

1 2 3 4 5 6 7 8 9 10

>> B = linspace (30 , 50 , 21)

B =

30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50

* **Length (A)** : Bu komut ilgilenilen bir vektörün uzunluğunu (*elemanlarının sayısı*) belirterek geri döndürür.

ÖRN : >> A = 2 : 3 : 45 ; length (A)

% ans = 2 5 8 11 14 17 20 23 26 29 32 35 38 41 44

ans =

15

ÖRN :

>> t=[1 2 3 4 5 6 7 8 9]

t =

1 2 3 4 5 6 7 8 9

>> z=[1,2,3,4,5,6,7,8,9,]

z =

1 2 3 4 5 6 7 8 9

>> length(z)

ans =

9

>> length(t)

ans =

9

veya ;

length (t)

disp ('bu t')

length (z)

disp (' bu z)

```
ans = % command window da görünümü
9
bu t
ans =
9
bu z
```

* Cebirsel işlemler , matematiksel kurallar içerisinde , MATLAB 'daki vektörlere uygulanabilir. Vektörler üzerinde işlem yapan diğer önemli fonksiyon nokta (*skaler*) ve (*vektörel*) çarpımlardır.Bunlar ;

* **>> dot (a , b)** : Skaler çarpım

ÖRN : **>> A = [4 7 3] , B = [-3 1 -5]**
A =
4 7 3
B =
-3 1 -5
>> dot (A , B) *%(4 * -3 + 7 * 1 + 3 * (- 5))*
ans =
20

* **>> cross (a , b)** : Vektörel çarpım

ÖRN :
$$\begin{matrix} a & b & c & 1 & 2 & 3 \\ a = d & e & f & , b = 4 & 5 & 6 \\ g & g & h & 7 & 8 & 9 \end{matrix}$$
 *% a*1+d*2+g*3.....*

* **norm (a , b)** : şiddetini bulur.

ÖRN : **>> a = [1 2 3]**
a =
1 2 3
>> norm (a) *% norm (şiddeti) = $\sqrt[3]{1^2 2^2 3^2} = \sqrt[3]{14} = 3,7417$*
ans =
3,7417

MATRİSLER :

* Matrisler, elemanları iki indisle tanımlanan iki boyutlu diziler olarak düşünülebilir.MATLAB 'da bir matrisin elemanları,bir köşeli parantez içinde her satırı noktalı virgüllerle ayırarak veya satır satır yazılarak girilebilir.

* Bir A matrisinin **i. ve j.** sütunundaki eleman **A(i , j)** şeklinde gösterilir.

* $A_{3 \times 3}$ matrisi, MATLAB 'da aşağıdaki gibi tanımlanır.

```
1 2 3
A = 4 5 6
7 8 9
```

1-); İşareti ile satır ayrımı yapılır ;

ÖRN : **>> A = [1 2 3 ; 4 5 6 ; 7 8 9]**
A =
1 2 3

```
4 5 6
7 8 9
```

2-) İkinci satır ve üçüncü sütundaki elemanı bulmak için ;

ÖRN : >> A(2 , 3)

```
ans =
1 2 3
4 5 6
7 8 9
6
```

3-) A Matrisinin birinci sütunundaki tüm elemanları çağırmak için ;

ÖRN : >> A(: , 1)

```
ans =
1
4
7
```

4-) A Matrisinin ikinci satırdaki tüm elemanları ;

ÖRN : >> A (2 , :)

```
ans =
4 5 6
```

5-) A matrisinin ikinci satırındaki tüm elemanları üç ile çarpmak için ;

ÖRN : >> A (2 , :) * 3

```
ans =
12 15 18
(4*3) (5*3) (6*3)
```

6-) Her satırdan sonra giriş tuşu ile de satır ayrımı yapılabilir ;

ÖRN : >> A = [1 2 3

```
4 5 6
7 8 9 ]
```

```
A =
1 2 3
4 5 6
7 8 9
```

7-) 2.Satır ve 3. sütun elemanı ile 1. ve 2. sütun elemanlarının çarpımı

ÖRN : >> A(2 , 3) * A (1 , 2)

```
ans =
12
%( 6 * 2 )
```

8-) >> B=[0;0;6]

```
B =
0
0
6
```

* a(:) : a matrisinin sütunlarının ard arda dizilmesinden oluşan bir sütun vektör oluşturur.

* **a(:,i)** : a matrisinin i. sütununu alır.

* **a(j,:)** : a matrisinin j. satırını alır.

* **a(:,[i j])** : a matrisinin i ve j sütununu alır.

* **a([i j],:)** : a matrisinin i ve j satırını alır.

* **a(:,i) = []** : a'nın i. sütununu siler.

* **a(i,:) = []** : a'nın i. satırını siler.

* Matlab ' da bir kısmı aşağıda verilen matris üreten bazı hazır fonksiyonlar da vardır.

* **>> eye (m , n)** : Birim matris

```
1 0 0
0 1 0
0 0 1
```

* **>> ones (m n)** : Birler matrisi

```
1 1 1
1 1 1
1 1 1
```

* **>> zeros (m n)** : Sıfırlar matrisi

```
0 0 0
0 0 0
0 0 0
```

ÖRN :

>> a=zeros(4)

```
a =
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
```

>> b=zeros(2,3)

```
b=
0 0 0
0 0 0
```

* **>> rand (m n)** : Rastgele sayılar matrisi

```
5 9 7
4 3 6
2 5 6
```

* **trace (a)** : Bir a matrisinin izini (köşegen elemanlarının toplamını) hesaplar.

* **diag (a)** : Bir kare a matrisinin köşegen elemanlarını bir sütun vektöre atar yada a bir vektör ise köşegenleri bu vektörün elemanlarından oluşan bir köşegen matris oluşturur.

* MATLAB ' da matrisler üzerinde bütün matematiksel işlemler gerçekleştirilebilir. (*matematiksel kurallar içerisinde*). Temel matris işlemlerinden bazıları aşağıdaki tabloda verilmiştir.

SİMGE	İŞLEM	AÇIKLAMA
+	$a + b$	Toplama
-	$a - b$	Çıkarma
*	$a * b$	Çarpma
/	$a / b (= a * b^{-1})$	Sağa bölme
\	$a \backslash b (= a^{-1} * b)$	Sola bölme
^	$a ^ b$	Üs alma
=	$a = b$	Atama
.*	$a .* b$	Eleman elemana çarpma
./	$a ./ b$	Eleman elemana sağa bölme
.\	$a .\ b$	Eleman elemana sola bölme
.^	$a .^ b$	Eleman elemana üs alma
'	a'	transpozu (evriğini) alma

ÖRN : $A = \begin{bmatrix} 1 & 2 \\ 3 & -2 \end{bmatrix}, B = \begin{bmatrix} 0 & 2 \\ 1 & 3 \end{bmatrix}$

1-) Matlab ' a yazılışı ;

```
>> A = [ 1 2 ; 3 -2 ], B = [ 0 2 ; 1 3 ]
```

A =

1 2

3 -2

B =

0 2

1 3

2-) Matrislerin toplanması ;

```
>> A + B
```

ans =

1 4

4 1

3-) Eleman elemana kare işlemi ;

```
>> A.^2
```

ans =

% 1*1 2*2 3*3 (-2)*(-2)

1 4

9 4

4-) Matrislerin çarpımı ;

```
>> A * B
```

ans =

2 8

-2 0

5-) Eleman elemana çarpım ;

```
>> A .* B                                >> A ^ 2                                % Gerçekte A ^ 2 = A * A işlemidir
ans =                                     ans =
0    4                                    7    -2
3   -6                                    -3   10
>> A * B'
ans =
4    7
-4   -3
```

6-) Matriste bölme ;

```
>> A / B                                % A / B = A * inv( B ) işlemide aynı işlemidir.
ans =
-0.5000    1.0000
-5.5000    3.0000

>> B / A                                % B \ A = B * inv( A ) işlemide aynı işlemidir.
ans =
0.7500   -0.2500
1.3750   -0.1250

>> A \ B                                % A \ B = inv ( A ) * B
ans =
0.2500    1.2500
-0.1250    0.3750
```

7-) Eleman elemana bölme ;

```
>> A ./ B
ans =
0    1.0000
0.3333   -1.5000
```

* Matrislerde eleman elemana işleme için ihtiyaç duyulduğunu göstermek üzere bir parçacığın konumunun zamana göre değişimini veren aşağıdaki fonksiyonu göz önüne alalım :

```
ÖRN : x ( t ) = t3 + 2t2 + 3                                % Burada t = saniye , x = metre cinsindendir.
                                                                Bu süreç MATLAB' DA aşağıdaki gibidir.
>> t = linspace ( 0 , 10 , 50 ) ;                                % " Zaman " vektörünü üret
>> x = t ^ 3 + 2 * t ^ 2 + 3                                % Noktalar atlanmış !!!
Error using ^                                                % Hatası veriyor.
Inputs must be a scalar and a square matrix.
To compute elementwise POWER, use POWER (.^) instead.
>> x = t .^ 3 + 2 * t .^ 2 + 3 ;                                % Eleman elemana işlem için noktalar kullanılmıştır.
x =
```

1.0e+03 *

Columns 1 through 12

0.0030 0.0031 0.0034 0.0040 0.0049 0.0061 0.0078 0.0100 0.0127 0.0159

0.0198 0.0244

Columns 13 through 24

0.0297 0.0358 0.0427 0.0504 0.0591 0.0688 0.0796 0.0914 0.1043 0.1185

0.1338 0.1505

Columns 25 through 36

0.1685 0.1879 0.2087 0.2310 0.2549 0.2804 0.3075 0.3363 0.3668 0.3992

0.4334 0.4695

.....

* Matlab ' da bir matrisin determinantını hesaplama , tersini bulma gibi özel matris işlemleri içinde hazır fonksiyonlar vardır.Bu fonksiyonlardan bazıları aşağıdaki gibidir.

FONKSİYON	MATLAB KULLANIMI
Bir matrisin determinantını hesaplar	det (A)
Bir matrisin tersini hesaplar	inv (A)
Bir matrisin özdeğerleri ve özvektörlerini bulur.	[v , d]= erg (A)

* Matrisler / Vektörlerin veri analizinde kullanılabilecek bazı fonksiyonlar aşağıdaki gibidir.

FONKSİYON	MATLAB KULLANIMI
Bir dizi en büyük değerini alır	max (A)
Bir dizinin en küçük değerini bulur	min (A)
Bir dizinin toplamını bulur	sum (A)
Bir dizinin aritmetik ortalamasını bulur	mean (A)
Bir diziyi azalan düzene göre sıralar	sort (A)
Bir dizinin boyutunu verir	size (A)
Vektörün uzunluğunu verir	length (A)
Alt üçgensel oluşturur	triu (A)
Üst üçgensel oluşturur	tril (A)

ÖRN: >> A = [1 2 3 ; 3 -2 2 ; 3 5 6] , B = [0 2 1 ; 1 3 3 ; 7 5 2]

A =

1 2 3
3 -2 2
3 5 6

B =

0 2 1
1 3 3
7 5 2

ÖRN: >> triu(A)

ans =

1 2 3
0 -2 2
0 0 6

ÖRN: >> tril(B)

ans =

0 0 0

```
1 3 0
7 5 2
```

*** >> reshape (A,m,n)** : Matlab’da A(mxn) boyutunda bir matris $m*n = p*q$ olmak şartıyla B(pxq) boyutunda bir matrise dönüştürülebilir.

ÖRN : A matrisi (2x3) boyutlarındadır. A matrisini (3x2) boyutuna getiriniz.

```
>> A = [10 8 6 ; 1 3 5]
```

```
A =
```

```
10 8 6
```

```
1 3 5
```

```
>> B=A
```

```
B =
```

```
10 8 6
```

```
1 3 5
```

```
>> C=reshape(A,3,2)
```

```
C =
```

```
10 3
```

```
1 6
```

```
8 5
```

UYGULAMA SORU VE CEVAPLARI 1 :

Aşağıdaki A ve B matrislerini göz önünde bulundurarak ;

$$A = \begin{bmatrix} 1 & 3 & 5 \\ 7 & 8 & 11 \\ 100 & 1 & 4 \end{bmatrix}, B = \begin{bmatrix} 10 & 5 & -5 \\ 70 & 8 & 7 \\ 10 & 1 & 3 \end{bmatrix}$$

ÖRN : Verilen A matrisi için aşağıdaki işlemleri komut satırında yapınız.

1-) A matrisini giriniz.

```
>> A = [ 1 3 5 ; 7 8 11 ; 100 1 4 ], B = [ 10 5 -5 ; 70 8 7 ; 10 1 3 ]
```

```
A =
```

```
1 3 5
```

```
7 8 11
```

```
100 1 4
```

```
B =  
10  5 -5  
70  8  7  
10  1  3
```

2-) A matrisinin determinantını hesaplayınız.

```
>> det(A)  
ans =  
-728.0000
```

3-) A matrisinin tersini bulunuz.Çıkan sonucu bir B matrisine atayınız.

```
>> inv (A)  
ans =  
-0.0288  0.0096  0.0096  
-1.4725  0.6813 -0.0330  
1.0893 -0.4107  0.0179  
>> B=inv(A)  
B =  
-0.0288  0.0096  0.0096  
-1.4725  0.6813 -0.0330  
1.0893 -0.4107  0.0179
```

4-) A * B işlemini yapınız.

```
>> A*B  
ans =  
1.0000      0  0.0000  
0  1.0000  0.0000  
0  0.0000  1.0000
```

5-) A matrisinin 1. sütununu a1 , 3. sütununu a3 vektörlerine atayınız.

```
>> a1=A(:,1) , a3=A(:,3)  
a1 =  
1  
7  
100  
a3 =  
5  
11  
4
```

6-) Köşegenleri A matrisinin köşegenlerinden oluşan bir C köşegen matrisi oluşturunuz.

```
>> C = [ 1 2 5 ; 4 8 3 ; 100 8 4 ]  
C =  
1  2  5  
4  8  3  
100 8  4
```

7-) a1' in devriği ile a3 vektörünü çarpınız.

```
>> a1'*a3  
ans =  
482
```

8-) a1 ile a3 vektör elemanlarını karşılıklı çarpınız.

```
>> a1.*a3
```

```
ans =  
5  
77  
400
```

9-) A' nın 3. satırını , diğer satır elemanlarını girmeden [5 6 7] olarak değiştir.

```
>> A(3,:)= [5 6 7]  
A =  
1   3   5  
7   8  11  
5   6   7
```

10-) A' nın 1 ve 2. satırlarını siliniz.

```
>> A(1,:)= []  
A =  
7   8  11  
100  1   4  
>> A(2,:)= []  
A =  
7   8  11
```

UYGULAMA SORU VE CEVAPLARI 2 :

* Verilen a matrisi için aşağıdaki işlemleri komut satırında (command window'da) yapınız.

1-) a matrisini giriniz.

```
>> a=[1 3 4 5 ; 4 2 7 5 ; 8 4 2 6]  
a =  
1   3   4   5  
4   2   7   5  
8   4   2   6
```

2-) a matrisini mevcut çalışma klasörünüze katsayilar ismiyle kaydediniz.

```
>> save katsayilar
```

3-) Dosyanın kaydedilip kaydedilmediğini kontrol ediniz.

```
open files sekmesinden
```

4-) MATLAB oturumundaki tüm değişkenleri siliniz.

```
clear all
```

5-) Command window'da yazılmış tüm ifadeleri temizleyiniz.

```
clc
```

6-) $a \times 2$ işlemini yapınız.

Undefined function or variable 'a'.

7-) a matrisini geri çağırınız.

```
load katsayilar
```

8-) a matrisinin üst ve alt üçgen matrislerini oluşturunuz.

```
>> triu(a)
```

```
ans =
```

```
1 3 4 5
```

```
0 2 7 5
```

```
0 0 2 6
```

```
>> tril(a)
```

```
ans =
```

```
1 0 0 0
```

```
4 2 0 0
```

```
8 4 2 0
```

9-) $c = [a \text{ zeros}(3, 2)]$ işlemini yapınız.

```
>> c = [a zeros(3, 2)]
```

```
c =
```

```
1 3 4 5 0 0
```

```
4 2 7 5 0 0
```

```
8 4 2 6 0 0
```

MATLAB 'DA GRAFİK ÇİZME:

* MATLAB çok boyutlu verileri , istenen formatta grafiksel olarak göstermeye yarayan hazır fonksiyonlar içerir.

* Grafiksel gösterim için kullanılan bazı hazır fonksiyonlar aşağıdaki tabloda verilmiştir.

FONKSİYON	AÇIKLAMASI	MATLAB KULLANIMI
plot	İki boyutlu çizim için temel komut	plot(xdata , ydata)
polar	Kutupsal (polar) koordinatlardaki çizim	polar(theta , rho)
plot3	Üç boyutlu (3D) çizimler	plot3(x , y , z)
title	Grafiğin üstüne adını yazmak için	title('myplot')
xlabel , ylabel	X eksenine ve Y eksenine etiket yazar	xlabel(' xdata ') , ylabel('ydata')

<i>grid</i>	<i>Grafiği doğru ağılarıyla örür</i>	<i>grid</i>
<i>subplot</i>	<i>Grafik penceresini bölmelere ayırır</i>	<i>subplot(m n k)</i> <i>% m=satır , n=sutun , k=bölmedeki yeri</i>
<i>text</i>	<i>grafikte istenen yere bit metin yerleştirir</i>	<i>text(x ,y , 'aplot')</i>
<i>gtext</i>	<i>Grafikte istenen noktaya bir metin yerleştirir</i>	<i>gtext('my plot')</i>
<i>ginput</i>	<i>Ekran üzerinde istenilen koordinatlardaki noktaları belirtmede kullanılır.</i>	<i>ginput</i>
<i>axis</i>	<i>Çoklu grafik çiziminde farklı grafikleri etiketler</i>	<i>legend('name1','name2')</i>
<i>Hold</i>	<i>Mevcut çizimi tutar</i>	<i>hold-hold on-hold off</i>
<i>linspace</i>	<i>Çizgi kalınlığı</i>	
<i>Data cursor</i>	<i>Grafik üzerindeki çizgide herhangi bir yeri tıkladığımızda o değeri gösterir.</i>	
<i>Axis</i>	<i>Grafikte hangi değerler arası görünsün istiyorsan yaz.</i>	<i>axis ([x₁x₂y₁y₂])</i>
<i>Stem</i>	<i>Farklı şekillerde çizme işlemi sağlar.</i>	<i>Stem (x,y)</i>
<i>legend</i>	<i>Grafiği isimlendirme</i>	<i>Legend('y1 grafiği','y2 grafiği','y3 grafiği')</i>
<i>linewidth</i>	<i>Çizgileri kalınlaştırır.</i>	<i>(x,y1,'-b','linewidth',2) Buradaki ' 2' çizgi kalınlığının kademesidir</i>

* Grafik üzerinde aynı anda 2 değer göremek istersen 1. değerin üzerine sağ tıklayıp *Create New Datatip* 'e tıkla ve 2. değeri seçin.

ÖRN : Aşağıda çizim ve etiketleme fonksiyonlarının kullanımına ilişkin bazı komutlar ve bu komutların görevleri açıklanmıştır.

```
>> t= 0 : 0.01 : 2 * pi ; x = sin ( t ) ; y = cos ( t ) ;
```

```
>> plot ( t , x )
```

```
>> grid
```

```
>> xlabel ( ' t ' )
```

```
>> hold
```

```
>> plot ( t , y )
```

```
>> ylabel ( ' x and y ' )
```

```
>> title ( ' sine and cosine curves ' )
```

% t ' ye göre x' i çizer.

% taksimat çizgilerini çizer.

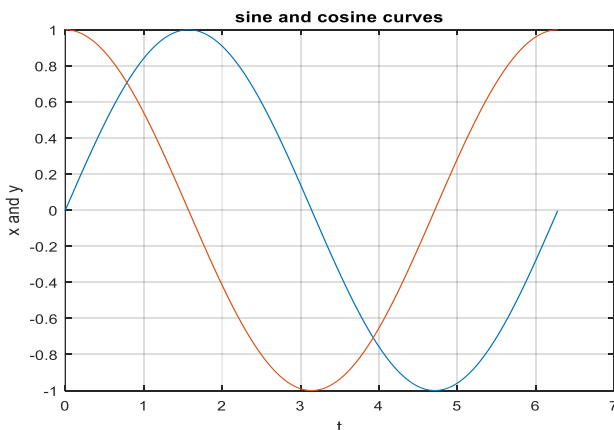
% yatay (x) eksenine " t " etiketini yazar.

% çizimi tutar ; yani yeni çizimi üzerine yazar.

% t ' ye göre y ' yi çizer.

% düşey eksene " x ve y " yazar

% grafiğe ad basar.



* Aynı grafik penceresinde bulunan eğrilerin karışmaması için, her eğri için farklı renkler ve farklı çizgi tipleri ile gösterilebilmektedir.

* Bazıları aşağıdaki tabloda verilmiştir.

RENK	SİMGE	ÇİZGİ TİPİ	SİMGE
Sarı	y	solid line	-
Pembe (Magenta)	m	dashed	--
Cam göbeği (Cyan)	c	dotted	:
Kırmızı	r	star	*
Yeşil	g	cicle	o
Beyaz	w	dash-dot	-.
Siyah	k	square	S

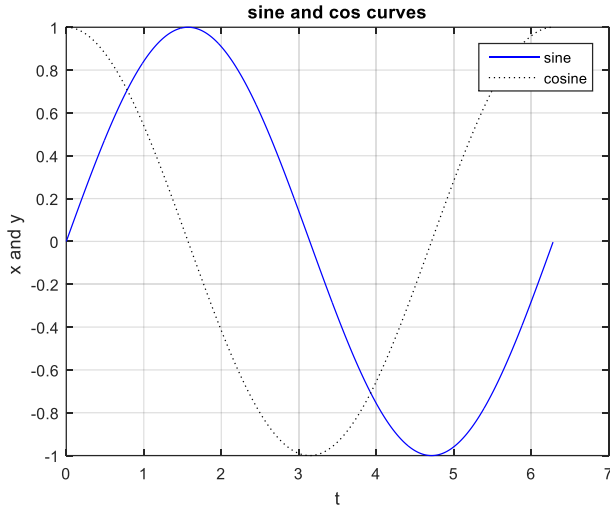
ÖRN : Yukarıdaki sin ve cos örneğinin birde renkli çizgilerle yazalım.

```
>> plot ( t, x, 'b', t, y, 'k:')
```

% Aynı şekil üzerinde mavi sürekli çizgiyle

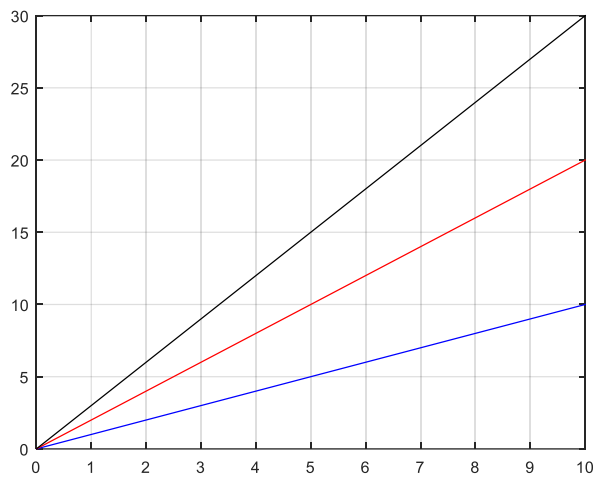
% x ' in t ' ye göre ve yine siyah kesikli çizgiyle y ' nin t ' ye göre değişiminin çizdirilmesi

```
>> grid
>> xlabel ( 't ')
>> ylabel ( 'x and y ')
>> title ( 'sine and cosine curves ')
>> legend ( 'sine ', 'cosine ')
```



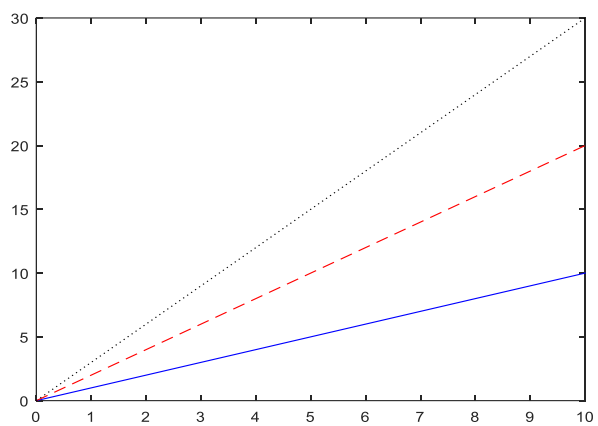
ÖRN :

```
>> x=0:1:10;
>> y1=x;
>> y2=2*x;
>> y3=3*x;
>> plot(x,y1,'-b') %blue
>> hold on
>> plot(x,y2,'-r') %red
>> hold on
>> plot(x,y3,'-k')
>> grid
```



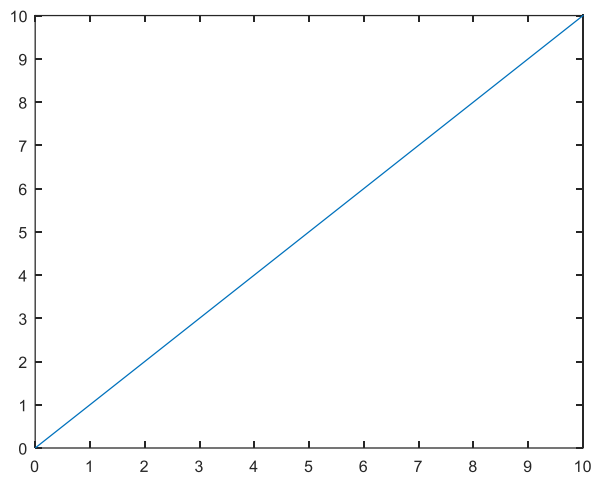
ÖRN:

```
>> x=0:1:10;
>> y1=x;
>> y2=2*x;
>> y3=3*x;
>> plot(x,y1,'-b')
>> hold on
>> plot(x,y2,'--r')
>> hold on
>> plot(x,y3,':k')
```



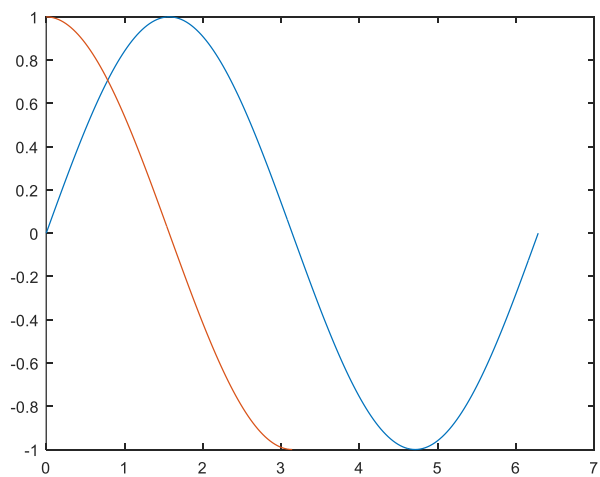
ÖRN :

```
>> x=0:1:10;
>> y=x
y =
0  1  2  3  4  5  6  7  8  9 10
>> plot(x,y)
```



ÖRN:

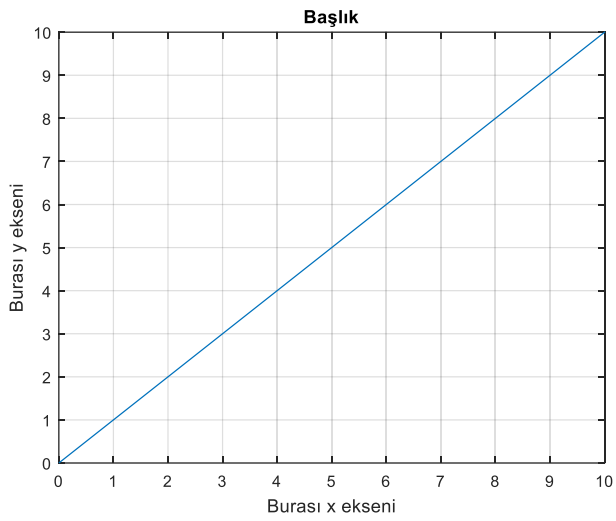
```
>> x1=0:pi/100:2*pi;
>> x2=0:pi/100:pi;
>> y1=sin(x1);
>> y2=cos(x2);
>> plot(x1,y1,x2,y2)
```



ÖRN :

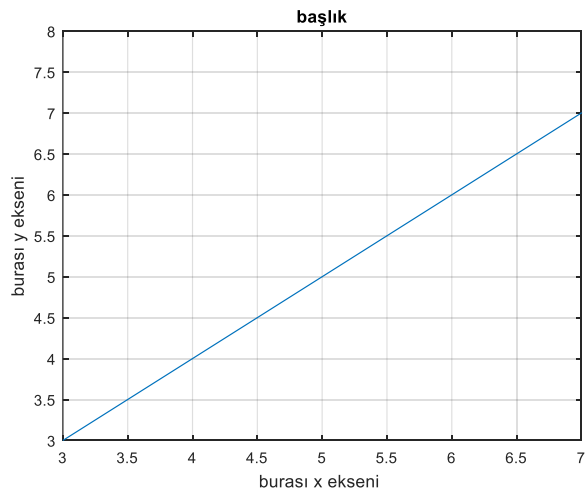
```
>> x=0:1:10;
>> y=x
y =
0  1  2  3  4  5  6  7  8  9 10
>> plot(x,y)
```

```
>> grid on
>> xlabel ('Burası x eksenı')
>> ylabel ('Burası y eksenı')
>> title('Başlık')
```



ÖRN :

```
>> x= 0 : 1 : 10 ;
>> y=x ;
>> plot ( x , y )
>> grid on
>> xlabel ('burası x eksenı ' )
>> ylabel (' burası y eksenı ' )
>> title ('başlık' )
>> axis ([ 3 7 3 7 ])
```

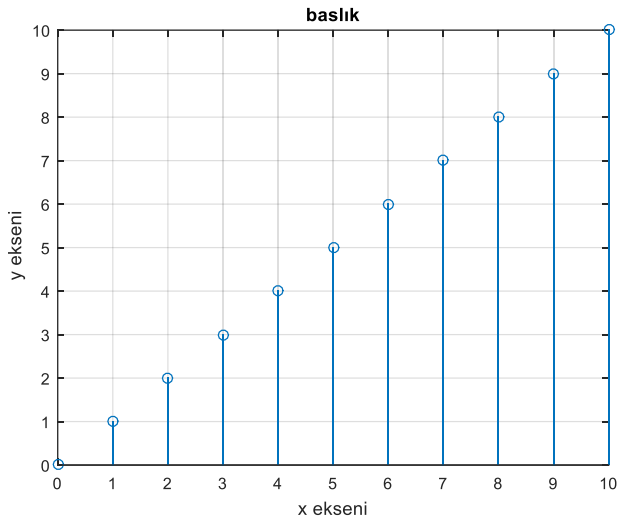


ÖRN :

```
>> x=0:1:10;
>> y=x
y =
0 1 2 3 4 5 6 7 8 9 10
>> xlabel ( ' x eksenı ' )
```

```
>> ylabel('y eksenini')
```

```
>> title('başlık')
```



ÖRN:

```
>> x=0:1:10;
```

```
>> y1=x;
```

```
>> y2=2*x;
```

```
>> y3=3*x;
```

```
>> plot(x,y1,'-b')
```

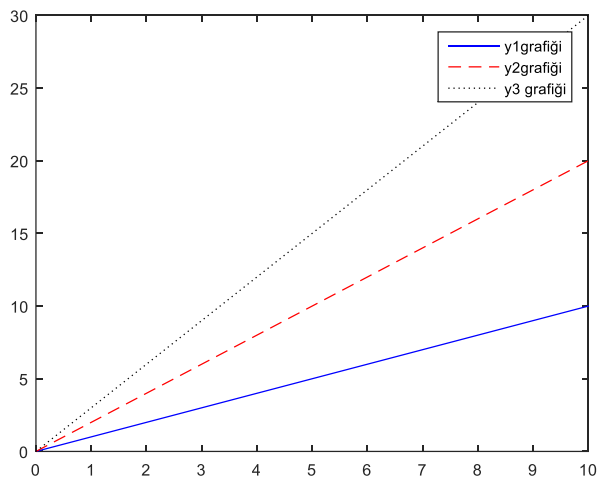
```
>> hold on
```

```
>> plot(x,y2,'--r')
```

```
>> hold on
```

```
>> plot(x,y3,':k')
```

```
>> legend('y1 grafiği', 'y2 grafiği', 'y3 grafiği')
```



ÖRN :

```
>> x=0:1:10;
```

```
>> y1=x;
```

```
>> y2=2*x;
```

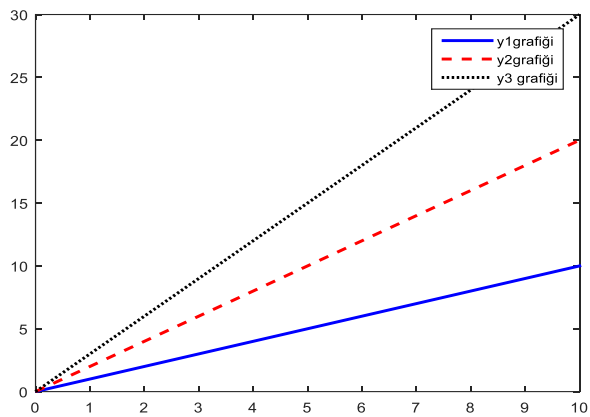
```
>> y3=3*x;
```

```
>> plot(x,y1,'-b','linewidth',2)
```

```

>> hold on
>> plot(x,y2,'-r','linewidth',2)
>> hold on
>> plot(x,y3,':k','linewidth',2)
>> legend('y1 grafiği', 'y2 grafiği', 'y3 grafiği')

```

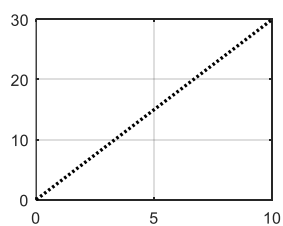
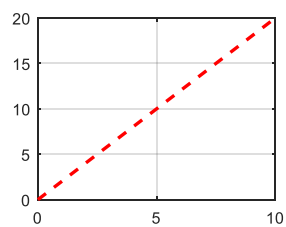
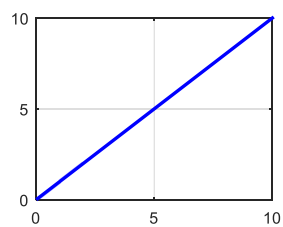


ÖRN:

```

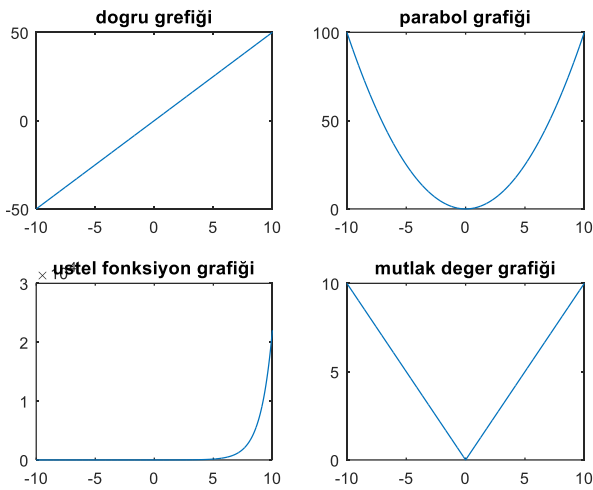
>> x=0:1:10;
y1=x;
y2=2*x;
y3=3*x;
subplot(2,2,1)
plot(x,y1,'-b','linewidth',2)
grid
hold on
subplot(2,2,2)
plot(x,y2,'-r','linewidth',2)
grid
hold on
subplot(2,2,3)
plot(x,y3,':k','linewidth',2)
grid

```



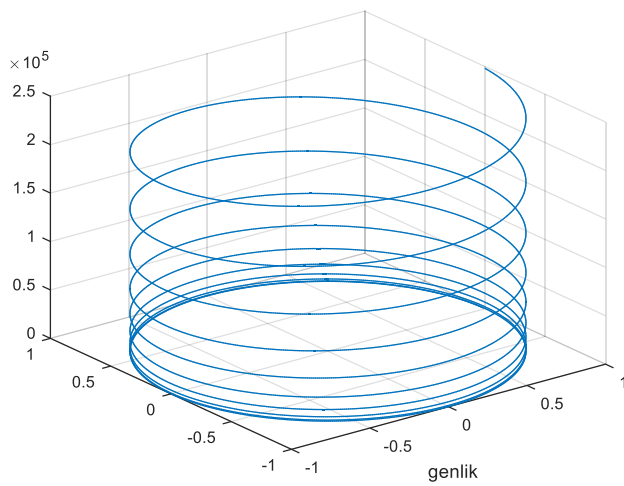
ÖRN : $y=5x$, $y=x^2$, $y=e^x$ ve $y=|x|$ fonksiyonlarını 2 satır 2 sütun şeklinde aynı figure penceresinde - $10 \leq x \leq 10$ aralığında 0.01 artımla çizdirin.

```
>> x=-10:0.01:10;
>> dogru=5*x;
>> parabol=x.^2;
>> ustel=exp(x);
>> mutlak=abs(x);
>> subplot(2,2,1)
>> plot(x,dogru)
>> title('dogru grafiği')
>> subplot(2,2,2)
>> plot(x,parabol)
>> title('parabol grafiği')
>> subplot(2,2,3)
>> plot(x,ustel)
>> title('ustel fonksiyon grafiği')
>> subplot(2,2,4)
>> plot(x,mutlak)
>> title('mutlak deger grafiği')
```



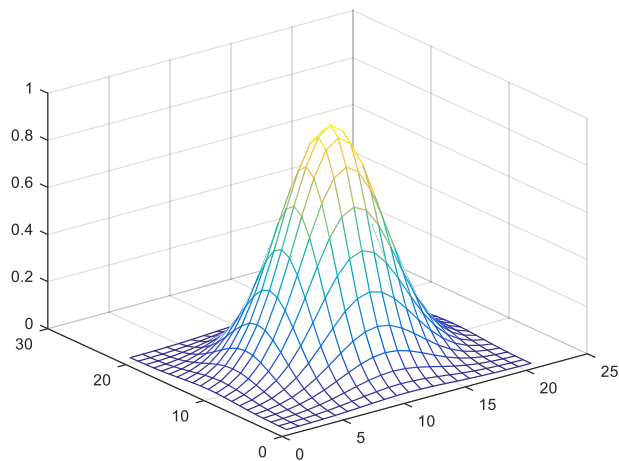
ÖRN :

```
>> t=0.01:0.01:20*pi;
>> x=cos(t);
>> y=sin(t);
>> z=t.^3;
>> plot3(x,y,z);
>> xlabel('helix');
>> xlabel('genlik');
>> grid
```



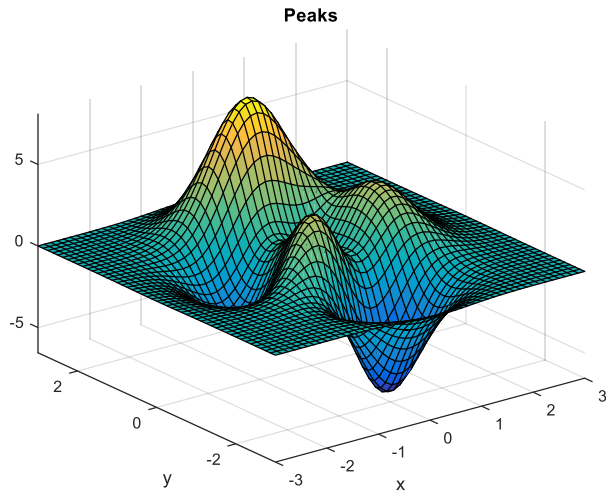
ÖRN:

```
>> x=-2:0.2:2;
>> y=x;
>> [x,y]=meshgrid(x,y);
>> z=exp(-x.^2-y.^2);
>> mesh(z)
```



ÖRN:

```
>> [X,Y]=meshgrid(-3:0.125:3);
>> peaks(X,Y)
z = 3*(1-x).^2.*exp(-(x.^2) - (y+1).^2) ...
- 10*(x/5 - x.^3 - y.^5). *exp(-x.^2-y.^2) ...
- 1/3*exp(-(x+1).^2 - y.^2)
```

Çizimden veri alma :

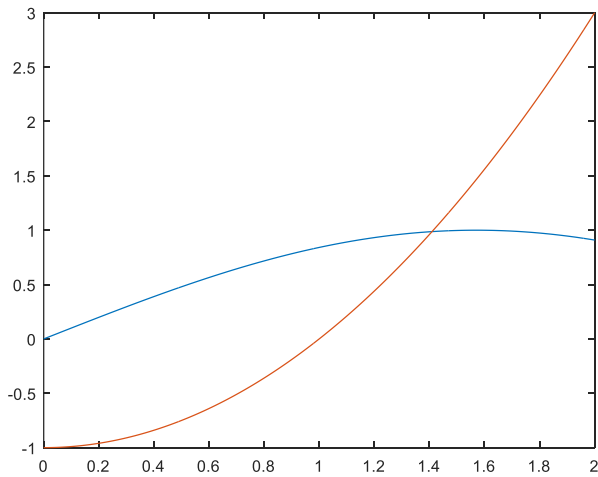
*** [x,y]=ginput(n)**

n : istenen nokta sayısı

x,y : Her noktaya karşılık gelen koordinat değerleri

ÖRN: $\sin(x)$ ve x^2-1 fonksiyonlarını $x=0-2$ aralığında çizdirerek kesiştikleri noktayı bulunuz.

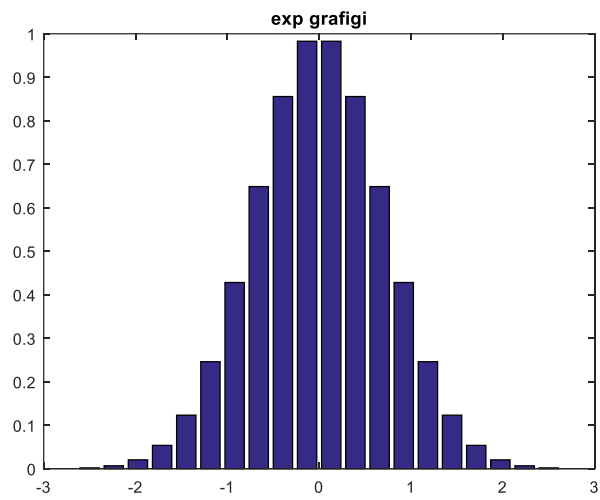
```
>> x=0:.01:2;
y1=sin(x); y2=x.^2-1;
plot(x,y1,x,y2)
[k1,k2]= ginput(1)
```



```
k1 =
1.4078
k2 =
0.9883
```

ÖRN:

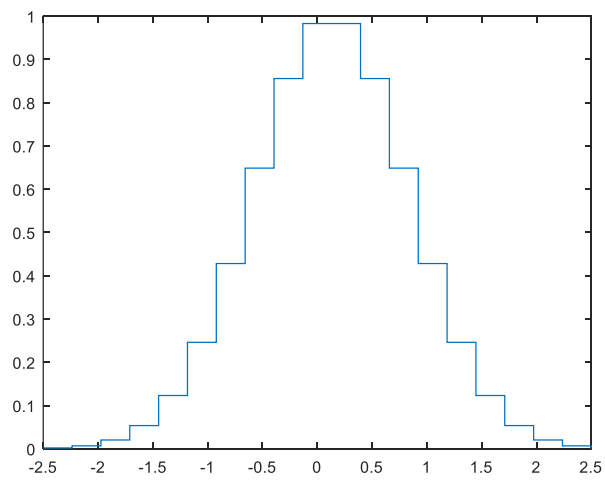
```
>> x=linspace(-2.5,2.5,20);
>> y=exp(-x.*x);
>> bar(x,y)
>> title('exp grafigi')
```



*** stairs(x,y)** : Merdiven şeklindeki çizim

ÖRN:

```
x=linspace(-2.5,2.5,20);
y=exp(-x.*x);
stairs(x,y)
title('exp grafigi')
```

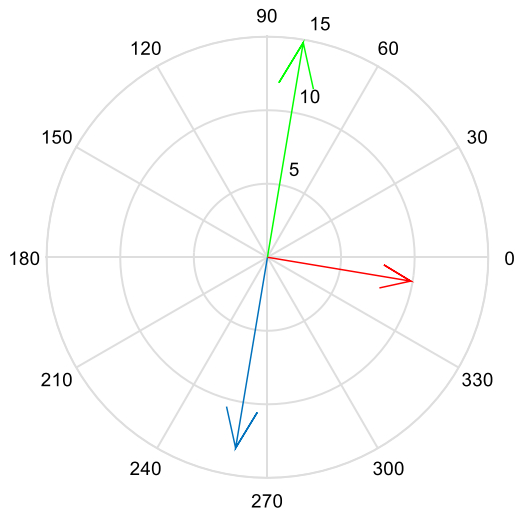


*** Compass(Z)** : Vektörel çizim yapar.

ÖRN:

$R=10$, $C=500\mu\text{F}$, $L=0.1\text{H}$ seri RLC devresinde $V_k=10\sin(150\pi t)$ ise devredeki V_c, V_L, V_r gerilimlerinin fazör diyagramını çiziniz.

```
>> R=10; C=500e-6;  
L=0.1;w=150;Vk=10;  
Z=R+i*(w*L-(1/(w*C)));  
I=Vk/Z;  
Vr=R*I;  
Vc=(1/(w*C*i))*I;  
VL=(w*L)*i*I;  
clf  
compass(Vc)  
hold on  
compass(Vr,'r')  
hold on  
>> compass(VL,'g')  
hold off
```



DÜZ YAZI DOSYALARI (SCRIPT FİLES)

MATLAB 'da kolay problemleri komut satırında tasarlamak ve uygulamak kolay olabilir. Ancak değişken ve yapılacaklar işlem sayısı arttıkça işlem sayısı arttığından her komutu teker teker yürütülmesi oldukça zordur. Bu durumu ortadan kaldırmak için yukarıdan aşağıya tasarım mantığıyla hazırlanan programlar kullanılır.

* Bu programların saklandığı dosyalar senaryo dosyası (**script file**) olarak adlandırılır. Bir senaryo dosyası (**script file**) özel bir görevi gerçekleştirmek için MATLAB komutlarının saklandığı bir metin programıdır.

* Komutlar dizisi bir dosyada saklanır ve daha sonra bunları komut penceresinden tek tek girmek yerine bu dosya çalıştırılarak komut dizisi yürütülür.

Bu dosyaların MATLAB ' ın çalıştığı birimde (**directory**) .m uzantısıyla saklanmaları gerekir. MATLAB senaryo dosyalarının oluşturulması ve yazılması için bir metin hazırlayıcısı (**text editor**) sunmaktadır. Bu senaryo dosyaları windows ' da notepad veya unix ' de vi -editor gibi herhangi bir metin hazırlayıcıda da yazılabilirler. MATLAB metin hazırlayıcısı ya komut penceresinin üst kısmında yer alan " **New M-file** " düğmesi tıklanarak veya kısaca " **File** " menüsünden " **New / M-file** " ibaresini seçerek etkin hale getirilebilir.

* Arşimet spiralini çizmek üzere aşağıda küçük bir dosya oluşturulmuş ve " **Archimedes.m** " adıyla saklanmıştır.

% Arşimet spiralini çizen bir M-dosyası (m- file)

% ($Rho = c * theta$ burada $c > 0$) [0 , 2 * pi] aralığında

% $c = 1$ kabul edelim

ÖRN :

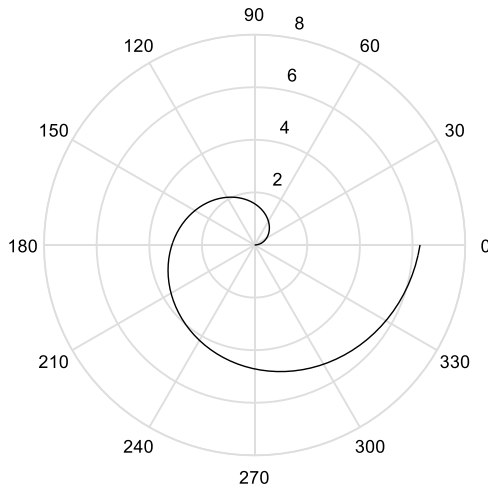
```
theta = linspace ( 0 , 2 * pi , 100 ) ;
```

```
c = 1 ;
```

```
Rho = c * theta ;
```

```
polar ( theta , Rho , ' k ' )
```

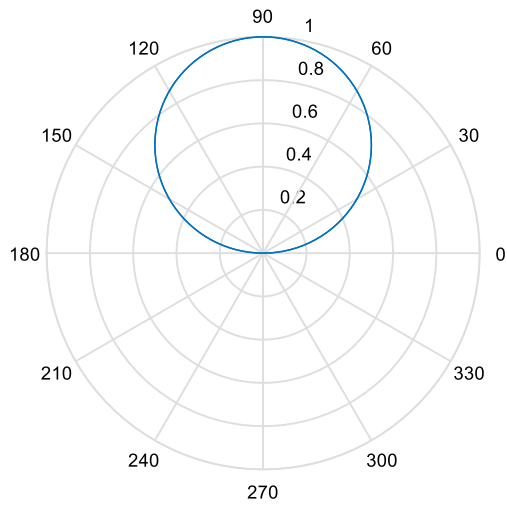
% [0 , 2 * pi] aralığında 100 nokta üretir.



ÖRN :

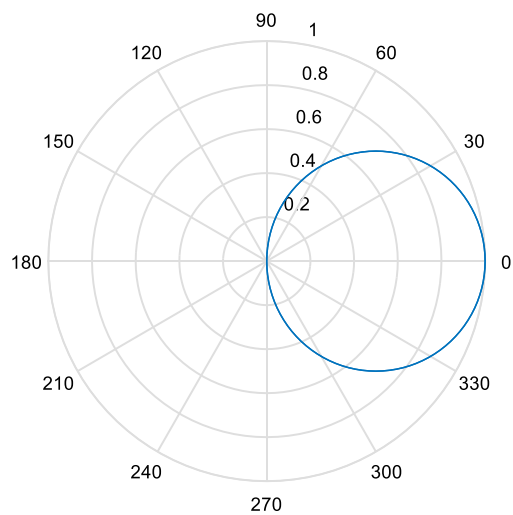
```
>> x=0:pi/100:2*pi;
```

```
>> length(x)
ans =
201
>> y=sin(x);
>> polar(x,y)
```



ÖRN :

```
>> x=0:pi/100:2*pi;
>> length(x)
ans =
201
>> y=cos(x);
>> polar(x,y)
```



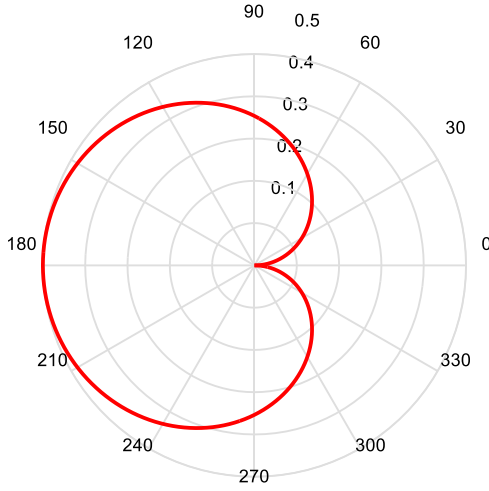
*** >> Set**

: İçeride değişiklik yapma için

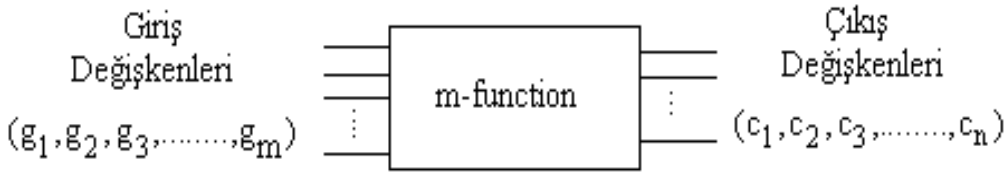
ÖRN :

```
x=0:pi/100:2*pi;  
y=0.5*sin(0.5*x);  
h=polar(x,y,'r')  
set(h,'linewidth',2)
```

% polarda çizgi rengi ('r'),('--r'),(':r')
% polardaki çizgi kalınlığı



FONKSİYON OLUŞTURMA



`[ c1,c2,,cn] = function dosya adı (g1,g2,....gm)`

* MATLAB fonksiyonları mathworks tarafından yazılmış bir takım senaryo dosyalarıdır. Senaryo dosyalarıyla (**script files**), fonksiyonları (**functions**) tanımlayan M-dosyaları arasındaki farklardan biri fonksiyon dosyalarının "**function**" terimi ile başlamasıdır. Bu terim , giriş ve çıkış parametrelerinin ne olacağı gibi , fonksiyonun formatını tanımlamaktadır. Dolayısıyla biri giriş (**input**) , diğeri çıkış (**output**) olmak üzere iki parametre listesine sahiptirler. Bir fonksiyon dosyasının ilk satırı aşağıdaki formatta belirtilmelidir ;

function [çıkış parametreleri] = fonksiyon_adi (giriş parametreleri)

* Burada **fonksiyon_adi** M-dosyasına verilen isimle aynı olmalıdır.

ÖRN : "**rad2deg**" adında bir fonksiyon oluşturulmuş ise , M-dosyasının adı da "**rad2deg.m**" olmalıdır. Giriş ve çıkış parametreleri skaler , vektörel ve / veya çok indisli diziler olabilir.

* Fonksiyonlardaki diğer bir kısıtlama da açıklama satırlarının "**function**" sözcüğüyle başlayan birinci satırdan sonra önlerine birer "**%**" işareti konarak yazılmak zorunda olmasıdır.

NOT : Aşağıda verilen örnek program radyan cinsinden verilen bir açıyı dereceye çeviren bir fonksiyon dosyasıdır.

Fonksiyon adı : **rad2deg.m** olduğu için;

M-dosyası : " rad2deg.m " adıyla saklanmalıdır.

ÖRN:

```
>> x0=fzero('exp(x)-sin(x)-3',0)
x0 =
1.3818
```

ÖRN :

function [deg] = rad2deg (rad) % rad2deg (rad) radyan cinsinden verilen bir açıyı dereceye çevirir.
rad * 180 / pi

```
>> rad2deg ( pi / 2 )
ans =
90
```

ÖRN :

```
function [y,x]=my_fun(x) % dosya adını my_fun olarak kaydet
y=x^2;
x=x+3;
end
>> x=2; [y,xp]=my_fun(x)
y =
4
xp =
5
```

ÖRN :

$x^2 + 2y^2 - 3 = 0$
 $4x^3 + 5x^2 + 6x + 1 - y = 0$

```
function F=my_fun(X)
a=1;
b=2;
r=3;
c=4;
d=5;
e=6;
f=1;
x=X(1);
y=X(2);
F(1)=a*x^2+b*y^2-r;
F(2)=c*x^3+d*x^2+e*x+f-y;
>> fsolve('my_fun',[1 1])
Equation solved.
```

fsolve completed because the vector of function values is near zero as measured by the default value of the function tolerance, and the problem appears regular as measured by the gradient.

```
<stopping criteria details>
ans =
0.0363 1.2245
```

* MATLAB' da fonksiyonlar ,fonksiyon dosyası kullanılmadan da oluşturulabilir.Dosya oluşturmaksızın fonksiyon tanımlamak için **inline** ve **isimsiz** fonksiyonlar kullanılır.

İNLİNE FONKSİYONLAR :

* **inline** fonksiyonlar , herhangi bir dosya oluşturmada fonksiyon tanımlamaya olanak sağlayan bir komuttur.

* inline fonksiyonlarda sadece tek bir ifade oluşturulabilir ve tek bir değişken geri döndürülebilir.

* Dolayısı ile mantıksal veya birden fazla işlemi gerektiren ifadeler için inline fonksiyonlar kullanılamaz.

* inline fonksiyonlar kullanıcı tarafından oluşturulan fonksiyon dosyasını çağırabilir.

Genel kullanımı ;

fonksiyon adı = inline ('fonksiyon ifadesi ' , 'x1 ' , 'x2 ' ...)

şeklindeBurada ;

fonksiyon adı : Fonksiyona verilmek istenen isim

fonksiyon ifadesi : Geçerli MATLAB ifadesi

x1 , x2 , ... : Fonksiyon ifadesindeki değişken isimleri

ÖRN :

```
>> fx : inline ( 'x ^ 2 * cos ( a * x ) - b ' , 'x ' , 'a ' , 'b ' ) ;
```

```
>> g = fx ( pi / 3 , 3 , 2 )
```

```
g =
```

```
-3.0966
```

İSİMSİZ (ANONYMOUS) FONKSİYONLAR :

* Anonim fonksiyonlar M-dosyaları veya alt fonksiyonu oluşturmak zorunda kalmadan , basit ifadeler için fonksiyon oluşturma araçlarıdır.

* Sadece bir ifade oluşturulabilir ve tek bir değişken geri döndürülebilir. Bu nedenle , mantık veya birden fazla işlemi gerektiren ifadeler için **anonim fonksiyonlar** kullanılamaz.Ancak **inline fonksiyondan** farklı olarak , isimsiz fonksiyon başka bir isimsiz fonksiyonu kullanabilir.

Genel kullanımı ;

fonksiyon adı = @ (x1 , x2 ,) (fonksiyon ifadesi)

Burada ;

fonksiyon adı : Fonksiyona verilmek istenen isim

fonksiyon ifadesi : Geçerli MATLAB ifadesi

x1 , x2 , ... : Fonksiyon ifadesindeki değişken isimleridir

ÖRN :

```
>> cx = @ ( x , b ) ( abs ( cos ( b * x ) ) ) ;
```

```
>> disp ( cx ( 4.1 , pi / 3 ) )
```

```
0.4067
```

DÖNGÜLER

* **ctrl+c** : döngüyü durdurur.

FOR DÖNGÜLERİ :

* Bir ifadeyi / işlemi tekrarlı olarak yürütmek için MATLAB ' ın sağladığı **for- döngüsü** ' nün yapısı ;

Genel yazımı ;

```
for
döngü_değişkeni = döngü_ifadesi
deyimler
end
```

döngü değişkeni : Bir değişkene verilen bir addır.

döngü ifadesi : Genel olarak kullanılacak indis değişkenine işaret eder .

(sayacın başlangıç , artım ve son değeri)

(**i = başlangıç değeri : artım : bitiş değeri**)

ÖRN :

```
function [ sum ] = sum1 (n)           % 1 ' den verilen bir n ' ye kadar tamsayıların toplamı
sum = 0 ;
for i = 1 : n                         % 1 ' den n ' ye 1 ' er artımla değişen indisi
sum = sum + i ;
end
>> sum1(4)
ans =
4
```

ÖRN:

```
a=1;
for i=1:3
for j=1:3
zzz(i,j)=a
a=a+1;
end
end                                % runa bas.
zzz =
1
zzz =
1  2
zzz =
1  2  3
zzz =
1  2  3
4  0  0
>> zzz(2,3)                       % 1-) 2= satır , 3= sütun kısmıdır.
ans=                                % 2-)Matristen hangi değeri görmek istiyorsan (a,b) a ve b yerine o değeri yaz.
6
```

NOT : Programı çalıştırmak için n ' nin yanında değişken adı neyse command window ' da o değişkeni yazıp değer verilecektir.

ÖRN:

```
>> k=1;
>> for i =1:1:3
```

```

>> for j= 1:1:3
>> z(i,j)=k;
>> k=k+1;
>> end
>> end
>> z
z =
1  2  3
4  5  6
7  8  9

```

WHİLE DÖNGÜLERİ :

* Tekrarlı deyimleri elde etmenin bir başka yolu " **while** " deyimini kullanmaktır. Bu durumda önceden verilen bir şart sağlamadığı sürece iterasyon süreci devam eder.

Genel yazımı ;

```

while ifade
    deyimler
end

```

* Yukarıdaki (sum1.m) fonksiyonunu **while** deyimiyile aşağıdaki gibi yeniden yazabilir.

ÖRN :

```

function [ sum ] = sum2 ( n )           % 1 'den verilen bir n' ye kadar tamsayıların toplamı
sum = 0 ; i=1 ;
while i <= n
    sum = sum + i;
    i=i+1 ;
end
>> sum3(3)
ans=
6

```

* **Rem (a,b)** : kalan demek

ÖRN:

```

>> rem(25,4)
ans=
1

```

ÖRN :

```

x=[3 4 6 7 9 11 12 14 15] % amacımız x'deki 3'e bölünebilen sayıları y'de toplar y=[3 6 9 12 15]
i=1;
j=1;
while i<=9
    if rem(x(i),3)==0
        y(j)=x(i);
        j=j+1;
    end
    i=i+1;
end
y

```

```

x =
  3  4  6  7  9 11 12 14 15
y =
  3  6  9 12 15

```

ÖRN :

```

>> i=1;
>> while i<11
>> x(i)=i;
>> i=i+1;
end
>> x
x =
  1  2  3  4  5  6  7  8  9 10

```

ÖRN :

```

>> i=1;
>> while i<=5
>> y(i)=i*3;
>> i=i+1;
>> end
>> y
y =
  3  6  9 12 15

```

ÖRN :

```

i=10;
while i>0
z(11-i)=i;
i=i-1;
end
z
z =
 10  9  8  7  6  5  4  3  2  1

```

ÖRN :

```

>> x=[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15]
x =
Columns 1 through 11
  1  2  3  4  5  6  7  8  9 10 11
Columns 12 through 15
 12 13 14 15
>> j=1;
>> for i=1:length(x)
if rem(x(i),3)==0
y(j)=x(i);
j=j+1;
if rem(x(i),2)==0
j=j-1;

```

```

end
end
end
>> y
y =
3 9 15

```

DÖNGÜLERDE CONTINUE VE BREAK DEYİMLERİNİN KULLANIMI :

Continue deyimi :

** Bu deyim for ve while ile kurulan döngülerde verilen koşul veya durum sağlandığında program akışını bir sonraki döngü değeri ile devam ettirir. Eğer iç içe for veya while kullanılmışsa continue ' nin geçtiği end ile sonlandırılan döngü atlanıp bir sonraki döngüye geçilir.*

ÖRN :

```

t = [ -5 5 -4 4 -3 3 -2 2 -1 1 ];
for k = 1 : length ( t )
    if t ( k ) < 0
        continue
    end
    t ( k ) = log10 ( t ( 1 , k ) );
end
>> t
t =
Columns 1 through 5
-5.0000  0.6990 -4.0000  0.6021 -3.0000
Columns 6 through 10
0.4771 -2.0000  0.3010 -1.0000    0

```

Break deyimi :

** Bu durum for ve while ile kurulan döngülerde verilen koşul veya durum sağlandığında program akışını orada keser. Eğer iç içe for veya while kullanılmışsa sadece break deyiminin geçtiği döngü sona erdirip dıştaki döngüye geçilir.*

ÖRN :

```

disp ( ' m , n ' )
for k = 1:10
    m = 100 - t ^ 3 ;
    if m < 0
        break
    end
    n = sqrt ( m );
    disp ( [ m n ] )
end
>> t =
4
>> break1
m,n

```

% t değerini belirle sonra run tuşuna bas

36 6
36 6
36 6
36 6
36 6
36 6

KOŞUL DEYİMLERİ

* Bütün programlama dillerinin en önemli özelliklerinden biri deyimlerin belirli koşullar altında yürütüldüğü dallanmalardır. MATLAB ' da diğer programlama dillerinde olduğu gibi " **if deyimleri** " ni kullanmaktadır.

İf deyimi :

if koşul
deyimler
end

ÖRN :

```
a= input ( ' Bir sayı giriniz : ' )  
if a < 50  
    sonuc = a / 5 ;  
end  
if a >= 50  
    sonuc = a * 5 ;  
end  
sonuc  
>> a =  
6  
sonuc =  
1.2000
```

ÖRN :

```
a=10.2424542;  
dummy=questdlg('Virgülden sonra kac hane verilsin?','sonuc','2 hane','3 hane','3 hane');  
switch dummy  
case{' 2 hane ' }  
    fprintf('%1.2f',a)  
case{'3 hane'}  
    fprintf('%1.3f',a)  
end
```

ÖRN :

```
>> x=8;  
>> y=3;  
>> if x>7  
    y=5;  
end  
>> y  
y =  
5
```

İç içe if yapısı :

Genel kullanımı ;

```
if koşul1
deyimler1
if koşul2
deyimler2
end
end
```

ÖRN :

```
a = input ( 'sayi giriniz : ' )
if a < 50
sonuc = a / 5 ;
if a >= 50
sonuc = a * 5 ;
end
end
sonuc                                     % run yap.
>> deneme1
sayi giriniz :3
a =
3
sonuc =
0.6000
```

İf else yapısı :

Genel kullanımı ;

```
if koşul
deyimler1
else
deyimler2
end
```

ÖRN :

```
a = input ( 'sayi giriniz : ' )
if a < 50
sonuc = a / 5 ;
else
sonuc = a * 5 ;
end
sonuc                                     % run yap
>> deneme2
sayi giriniz : 8
a =
8
sonuc =
1.6000
```

Else if yapısı :

Genel kullanımı ;

```
if koşul1
deyimler1
else if koşul2
deyimler2
else
deyimler3
end
```

ÖRN :

```
a = input ( 'sayı : ' )
sayı giriniz : 8
if a < 50
sonuc = a / 5 ;
else if a < 100
sonuc = a + 5 ;
else
sonuc = a * 5 ;
end
sonuc
```

```
>> deneme2
sayı giriniz : 8
a =
8
sonuc =
1.6000
```

ÖRN : Aşağıda örnek , a , b , c sabitleri verilmiş olmak kaydıyla $ax^2 + bx + c = 0$ denkleminin köklerini bulmak için koşul deyimleriyle kurulmuş bir programdır :

```
function [ kokler ] = quadrt ( a , b , c ) % a ^ 2 + b x + c = 0 dekleminin köklerini bulan program
d = b ^ 2 - 4 * a * c ;
if d == 0
kokler = - b / ( 2 * a ) ;
disp ( ' katlı kok : ' )
disp ( kokler )
else if d > 0
kokler = [ - b - sqrt ( d ) , - b + sqrt ( d ) ] ./ ( 2 * a ) ;
disp ( ' iki farklı kok : ' )
disp ( kokler )
else
kokler = [ - b - i * sqrt ( abs ( d ) ) , - b + i * sqrt ( abs ( d ) ) ] ./ ( 2 * a ) ;
disp ( ' iki kompleks kok : ' )
disp ( kokler )
end
```

ÖRN :

```

x=[1 2 3 4 5 6 7 8 9 10]    % Amacımız y= [ 1 3 5 7 9 ] , z=[ 2 4 6 8 10]
i=1;
j=1;                        % z array i için sayac
k=1;                        % y array i için sayac
while i<=10
if rem(x(i),2)==0
z(j)=x(i);
j=j+1;
else if rem(x(i),2)==1
y(k)=x(i);
k=k+1;
end
i=i+1;
end
y
z

```

DEĞERLERİN SAYISAL FORMATLARI

Sayıların formatı (kaç hane ve kaç ondalıkla gösterileceği gibi) belirlenebilir.

MATLAB KULLANIMI	AÇIKLAMA	ÖRNEK
Format short	Virgülden sonra 4 basamak gösterimi	3.1416
Format long	Virgülden sonra 14 basamak gösterimi	3.14159265358979
Format short e	5 basamaklı üstel gösterim	3.1616e+000
Format long e	Virgülden sonra 15 basamak gösterimi	3.141592653589793e+000
Format short g	Yuvarlanmış veya kayan noktalı gösterimden kısa olanı tercih eder	3.1416
Format long g	Yuvarlanmış veya kayan noktalı gösterimden uzun olanı tercih eder	3.14159265358979
Format rat	İki sayının oranı şeklinde gösterim	355 / 113
Format bank	Virgülden sonra 2 basamak gösterimi	3.14
Format hex	Hexadecimal gösterim	400921fb54442d18

* **fprintf fonksiyonu** : Program çıktısının formatlı olarak sunulmasını sağlayan komuttur.

Genel kullanımı ;

fprintf (' biçim ' , değer)

ÖRN :

fprintf (' pi sayısı = % 5.12f \n\n ' , pi)

MATLAB ' DA SEMBOLİK İŞLEMLER

* MATLAB ' da sayısal hesaplama ile matematiksel sembolik işlemlerin birleştirildiği " **Symbolic Math Toolbox** " adı verilen araç kutusu vardır.Bu araç kutusu sembolik matematiksel işlemlerin MATLAB ile gerçekleştirilmesine izin verir.Matematiksel olarak elle yapılan bağıntı türetme işlemleri bu araç kutusu ile gerçekleştirilebilir. " **Symbolic Math Toolbox** " da bulunan bazı fonksiyonlar aşağıda kısaca verilmiştir.

* >> **help sym** : Sembolik işlemlerde , herhangi bir işlem için tanımlanan fonksiyonlar hakkında yardım sağlayan komuttur.

ÖRN :

>> **help sym**/diff

* >> **syms** : Sembolik işlemler yapabilmek için kullanılacak değişkenlerin tanımlanmasında kullanılır.

ÖRN :

>> **syms** a b c x

>> $f = a * x^2 + b * x + c$; % (>> $f = \text{sym}('a * x^2 + b * x + c');$);)

* >> **findsym** : Bir fonksiyonda tanımlanan sembolik değişkenleri bulur.

ÖRN :

>> **findsym** (f)

ans =

a , b , c , x

* >> **pretty** :Sembolik ifadenin matematiksel yazım şekliyle ekranda görüntülenmesini sağlar.

ÖRN :

>> **pretty** (f)

$a x^2 + b x + c$

* >> **simplify** : Sembolik ifadenin daha sade şekilde görüntülenmesini sağlar.

ÖRN :

>> **simplify** ($\cos(x)^2 + \sin(x)^2 + \cos(x)$)

ans =

$\cos(x) + 1$

* >> **expand** : Kapalı polinom halindeki sembolik ifadelerin açık polinom halini bulur.

ÖRN :

>> **expand** ($(x + 2)^4$)

ans =

$x^4 + 8 * x^3 + 24 * x^2 + 32 * x + 16$

* >> **collect** : Matematiksel ifade içerisinde dağınık şekilde bulunan değişkenleri azalan üs değerine göre bir araya toplar.

ÖRN :

>> **collect** ($2 * x + 4 * x^3 - 5 * x^2 + 6 * x + 2 * x^2 - 10 * x$)

ans =

$4 * x^3 - 3 * x^2 - 2 * x$

* >> **factor** : Açık halde yazılmış bir polinom ifadesinin kapalı (eğer varsa) gösterimini bulur.

ÖRN :

```
>> y = factor ( x ^ 2 + 6 * x + 9 )
y =
[ x + 3 , x + 3 ]
```

* >> rats(x) : x'i bir kesir olarak temsil eder

ÖRN :

```
>> rats(1.5)
ans =
3/2
```

* >> primes (x) : x'den küçük asal sayıları bulur.

ÖRN :

```
primes(10)
ans =
2 3 5 7
```

* >> numden : Belirli bir matematiksel ifadenin pay ve paydasını bulur.

ÖRN :

```
>> syms x y ;
>> [ pay , payda ] = numden ( x / y + y / x )
pay =
x ^ 2 + y ^ 2
payda =
x * y
```

* >> coeffs : Bir matematiksel ifadenin belirlenen değişkenine göre artan üs düzeninde katsayılarını bulur.

ÖRN :

```
>> f = y ^ 3 - 2 * y ^ 2 + 3 * x - 4 ;
>> c = coeffs ( f , y )
c =
[ 3*x - 4, -2, 1]
```

Sembolik Değişiklik :

* >> subs : Tanımlanmış bir denklemde sembolik değişkenlerin başka bir sembolik değişkenle değişimini sağlar.

ÖRN :

```
>> f = exp ( a ) * x ^ 3 + log ( b ) * x ^ 2 + c ;
>> a = 2 ;
>> b = 3 ;
>> c = x ;
>> r = subs ( f )
r =
[ exp (2)*x^3 + log(3)*x^2 + 3*x - 4, exp(2)*x^3 + log(3)*x^2 - 2, exp(2)*x^3 + log(3)*x^2 + 1]
```

Sembolik Olarak Denklem Çözümü :

*** >> solve** : Cebirsel denklemlerin sembolik olarak çözümünü gerçekleştirir.

ÖRN :

```
>> [ A1 , A2 ] = solve ( ' 3 * A1 + 4 * A2 = 2 ' , ' -2 * A1 + 3 * A2 = 4 ' )  
A1 =  
-10/17  
A2 =  
16/17
```

ÖRN :

```
>> klm = solve ( ' 5 = 10 * ( 1 - exp ( -1 / klm ) ) ' )  
klm =  
1/log(2)
```

ÖRN :

```
>> double ( klm )  
ans =  
1.4427
```

ÖRN :

```
>> solve('x^2-x-6')  
ans =  
-2  
3
```

ÖRN :

```
>> syms a x b c  
>> d=a*x^2+b*x+c;  
>> kok=solve(d);  
>> pretty(kok)  
/  
| b + sqrt(b^2 - 4 a c) |  
| - ----- |  
| 2 a |  
| |  
| |  
| b - sqrt(b^2 - 4 a c) |  
| - ----- |  
| 2 a |  
\  
/
```

ÖRN :

```
>> [x,y]=solve('x+2*y=1','x-y=2')  
x =  
5/3  
y =  
-1/3
```

Sembolik Olarak Diferansiyel Denklemlerin Çözümü :

*** >> dsolve** : Diferansiyel denklemlerin sembolik olarak çözümünü gerçekleştirir.

Bu fonksiyon tanımlanan diferansiyel denklemin t 'ye göre **integralini** alır.

Genel kullanımı ;

dsolve (' denklem1 ' , ' denklem2 ' , ' denklem3 ' , ... , ' başlangıç koşulları ')

Diferansiyel denklemin türev mertebeleri :

Dx : Birinci mertebe türev
D2x : İkinci mertebe türev
D3x : Üçüncü mertebe türev
..... gibi devam eder.

ÖRN :

```
>> Y=dsolve ( ' D2y + 4 * Dy + 4 = t ' )  
Y =  
C3 - (17*t)/16 + t^2/8 + C4*exp(-4*t) + 17/64
```

*** Başlangıç değeri ile birlikte ;**

ÖRN :

```
>> y0 = [ 0 , -1.2 ] ;  
>> y = dsolve ( ' D2y + 4 * Dy + 4 = t ' , ' y( 0 ) = 0 ' , ' Dy ( 0 ) = -1.2 ' )  
y =  
(11*exp(-4*t))/320 - (17*t)/16 + t^2/8 - 11/320
```

Sembolik Olarak Türev Alma :

*** >> diff** : Matematiksel olarak tanımlanan bir bağıntının **türevini** hesaplar.

*** x-tanımlı türev değişkeni kabul edilir.**Eğer matematiksel ifadede x değişkeni yoksa , x ' e alfabetik olarak en yakın harf türev değişkeni kabul edilir.

ÖRN :

```
>> syms a b c x y z  
>> diff ( a * x ^ 3 - exp ( -2 * x ) )  
ans =  
2*exp(-2*x) + 3*a*x^2
```

ÖRN :

```
>> A = [ cos ( a * x ) , sin ( a * x ) ; -sin ( a * x ) , cos ( a * x ) ] ;  
>> dAdx = diff ( A )  
dAdx =  
[ -a*sin(a*x), a*cos(a*x)]  
[ -a*cos(a*x), -a*sin(a*x)]
```

ÖRN :

```
>> f = a * x ^ 2 - b * y + c * z ;
```

```
>> trvf = diff ( f )
```

```
trvf =
```

```
2*a*x
```

ÖRN :

```
>> f = - b * y + c * z ;
```

```
>> trvf = diff ( f )
```

```
trvf =
```

```
-b
```

ÖRN :

```
>> syms x a b
```

```
>> f=5*x^3+a*x^2+b*x+14;
```

```
>> s=diff(f);
```

```
>> s
```

```
s =
```

```
15*x^2 + 2*a*x + b
```

ÖRN :

```
>> syms x;
```

```
>> f=1/(1+5*cos(x));
```

```
>> s=diff(f,3);
```

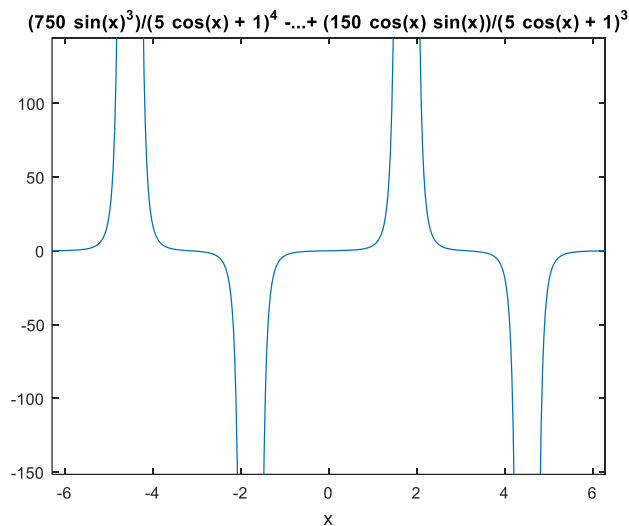
```
>> s
```

```
s =
```

```
(750*sin(x)^3)/(5*cos(x) + 1)^4 - (5*sin(x))/(5*cos(x) + 1)^2 + (150*cos(x)*sin(x))/(5*cos(x) + 1)^3
```

```
>> pretty(s)
```

$$\frac{750 \sin(x)^3}{(5 \cos(x) + 1)^4} - \frac{5 \sin(x)}{(5 \cos(x) + 1)^2} + \frac{150 \cos(x) \sin(x)}{(5 \cos(x) + 1)^3}$$



* Eğer matematiksel ifadenin türevinin belirlenen bir değişkene göre alınması isteniyorsa , bu değişken parametre olarak girilir.

ÖRN :

```
>> f = a * x ^ 2 - b * y + c * z ;
>> diff ( f , z )
ans =
c
```

* Eğer matematiksel ifadenin türevinin " n " kez alınması isteniyorsa , n değeri parametre olarak girilir.

ÖRN :

```
>> f = x ^ 6;
>> diff ( f , 6 )
ans =
720
```

ÖRN :

```
>> x= 0:0.1:30;
>> y=x.^6-4*x.^3 + 4*x-8*sin(x);
>> turev=diff(y)./diff(x)
turev =
1.0e+08 *
Columns 1 through 6
-0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000
Columns 7 through 12
-0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000
.....
```

Sembolik Olarak İntegral Alma :

* >> int : Matematiksel olarak tanımlanan bir bağıntının **sembolik integralini** hesaplar.

* x- değişkeni tanımlı integral değişkeni olarak kabul edilir.Eğer matematiksel ifadede x değişkeni yoksa , x ' e alfabetik olarak en yakın harf integral değişkeni kabul edilir.

ÖRN :

```
>> f = x ^ 3 + 3 * y - 2 * a ;
>> int ( f )
ans =
x^4/4 + (3*y - 2*a)*x
```

ÖRN :

```
>> f = y ^ 6 + 3 * a ;
>> int ( f )
ans =
y^7/7 + 3*a*y
```

* Eğer matematiksel ifadenin integralinin belirlenen bir değişkene göre alınması isteniyorsa , bu değişken parametre olarak girilir.

ÖRN :

```
>> f = x ^ 3 + 3 * y - 2 * a ;
```

```
>> int ( f , a )
ans =
a*(x^3 + 3*y) - a^2
```

* Eğer matematiksel ifadenin sınırlı integrali alınmak istiyorsa , sınırlı parametre olarak girilir.

ÖRN :

```
>> f = a * sin ( b * t );
>> int ( f , t , 0 , pi )
ans =
( 2*a*sin(pi*b)/2)^2)/b
```

* f, t : t ' ye göre türev alır.
* $0, pi$: sınırlar.

Sembolik Olarak Laplace Ve Ters Laplace Dönüşümü :

* >> **laplace** : Zaman alanında tanımlanmış periyodik bir fonksiyonun , **sembolik Laplace dönüşümünü** gerçekleştirir.Tanımlı zaman değişkeni -t , Laplace değişkeni-s olarak kullanılır.

ÖRN :

```
>> syms a t x
>> L = laplace ( t ^ 2 )
L =
2 / s ^ 3
```

ÖRN :

```
>> laplace ( exp ( -a * t ) , x )      % Tanımlı s değişkeni yerine x değişkeni kullanıldı.
ans =
1/(a + x)
```

* >> **ilaplace** : Laplace alanında tanımlanmış ifadenin sembolik olarak zaman alanına dönüşümünü gerçekleştirir.Tanımlı Laplace değişkeni-s , zaman değişkeni-t olarak kullanılır.

ÖRN :

```
>> ilaplace ( L )
ans =
t^2
```

ÖRN :

```
>> f = ilaplace ( 1 / ( t - a ) ^ 2 )
f =
x*exp(a*x)      % f ( s ) ' yi f ( t ) olarak kabul etti ve sonucu f ( t ) yerine f ( x ) olarak
```

döndürdü

Sembolik Olarak Limit Hesaplama :

* >> **limit** : Sembolik olarak tanımlanan bir ifadenin , belirlenen bir değişkene göre **limit** değerini hesaplar.Tanımlı değişken-x ve tanımlı değer olarak 0 kullanılır.

ÖRN :

```
lim f( x )  
x→0  
>> f = a * x ^ 2 + 2 * x - c  
>> limit ( f )  
ans =  
-c
```

* Eğer farklı bir değişken ve limit değeri kullanılacaksa , değişken ve değer parametre olarak girilir

ÖRN :

```
lim f( y )  
y→a  
>> limit ( f , y , 2 )  
ans =  
a*x^2 + 2*x - c
```

ÖRN :

```
>> syms x h  
>> dydx = limit ( ( cos ( x + h ) - cos ( x ) ) / h , h , 0 )  
dydx =  
-sin ( x )
```

$$\% f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

ÖRN :

```
lim  $\frac{x^3 - a^3}{\sin(3x - 3a)}$   
x→a  
>> syms x a  
>> w = (x^3 - a^3) / (sin(3*x - 3*a));  
>> limit(w, x, a)  
ans =  
a^2
```

Sembolik Taylor Serisi :

$$f(x) = \sum_{n=0}^{\infty} (x - a)^n \frac{f^{(n)}(a)}{n!}$$

* >> **taylor (f)** : " f " fonksiyonunun *beşinci dereceden Maclaurin seri açılımını* hesaplar.

* >> **taylor (f , v , a)** : v ' ye göre a noktası civarındaki f fonksiyonunun *Taylor seri* açılımını hesaplar.

ÖRN :

```
>> syms x  
>> f = sin ( x ) / x ;  
>> t6 = taylor ( f )  
t6 =  
x^4/120 - x^2/6 + 1
```

ÖRN :

```
>> t8 = taylor ( f , ' order ' , 8 )  
t8 =  
- x^6/5040 + x^4/120 - x^2/6 + 1
```

ÖRN :

```
>> taylor ( log ( x ) , x , 1 )  
ans =  
x - (x - 1)^2/2 + (x - 1)^3/3 - (x - 1)^4/4 + (x - 1)^5/5 - 1
```

MATLAB ' DA POLİNOMLAR

* MATLAB ' da satır vektörleri üs dereceleri azalan sırada olmak üzere , polinom katsayılarını temsil edebilir.

ÖRN :

$x^4 - 2x^2 + 3x^2 - 4x + 5 = 0$ denkleminin katsayıları komut satırında aşağıdaki gibi girilebilir.

```
>> p_x = [ 1 -2 3 -4 5 ]
```

```
p_x =
```

```
1 -2 3 -4 5
```

ÖRN:

$$y = f(X) = X^4 - 3 \cdot X^3 + 2 \cdot X^2 - 5 \cdot X - 11$$

```
>> p = [ 1, -3, 2, -5, -11]
```

** Sık kullanılan polinom fonksiyonları ve bu fonksiyonların tanımı aşağıda verilmiştir ;*

*** >> r = roots (p)** : Vektörel olarak tanımlanmış polinomun köklerini bulur ve sonucu bir kolon vektörü olarak geri döndürür.

ÖRN :

```
>> p_x = [ 1 -2 5 ];
```

```
>> roots ( p_x )
```

```
ans =
```

```
1.0000 + 2.0000i
```

```
1.0000 - 2.0000i
```

*** >> p = poly (r)** : Vektörü ile kökleri verilen bir polinomun , azalan üslü polinomun katsayılarını satır vektörü olarak elde eder. Her bir vektör için roots ve poly birbirinin ters fonksiyonlarıdır.

ÖRN :

```
>> p_x = [ 1 -2 5 ]; kokler = roots ( p_x)
```

```
kokler =
```

```
1.0000 + 2.0000i
```

```
1.0000 - 2.0000i
```

*** >> p = poly (A)** : A nxn boyutunda bir matris ise , $p = \text{poly} (A)$ fonksiyonu elemanları " $\det (sI - A)$ " karakteristik polinomun katsayıları olan n+1 elemanlı satır vektörünü hesaplar. Katsayılar azalan kuvvetlere göre sıralanır.

ÖRN :

```
>> A = [ 1 2 3 ; 4 5 6 ; 7 8 9 ]
```

```
A =
```

```
1 2 3
```

```
4 5 6
```

```
7 8 9
```

```
>> p = poly ( A )
```

```
p =
```

```
1.0000 -15.0000 -18.0000 -0.0000
```

```
>> r = roots ( p )
```

```
r = 16.1168
```

```
-1.1168
```

```
-0.0000
```

Not :

* $r = \text{roots}(\text{poly}(A))$ işlemi $\text{eig}(A)$ fonksiyonuna denktir.

* poly ve roots için kullanılan bu algoritma öz değer hesaplamak için farklı bir yaklaşımdır. $\text{poly}(A)$ fonksiyonu A 'nın karakteristik polinomunu oluşturur ve " $\text{roots}(\text{poly}(A))$ " A 'nın karakteristik denkleminin kökleri olan öz değerleri bulur.

* $\gg y = \text{polyval}(p, x)$: Fonksiyonu x verisi tarafından değerlendirilen polinomun değerini hesaplar. Burada " p " elemanları azalan kuvvetlere göre verilen polinomun katsayılarını içeren bir vektördür. " x " ise bir vektör veya matris olabilir. Herhangi bir durumda polyval fonksiyonu x 'in her bir elemanında p 'yi değerlendirir.

ÖRN :

```
>> p = [ 1 6 11 6 ];  
>> polyval ( p , 5 )  
ans =  
336
```

* $\gg q = \text{conv}(p1, p2)$: Fonksiyonu $p1$ ve $p2$ vektörlerini katlar. Matematiksel olarak konvolüsyon katsayıları $p1$ ve $p2$ 'nin elemanları olan polinomlarını çarpma ile aynı işleme karşılık gelir.

ÖRN :

```
>> p1 = [ 1 -2 1 ]; p2 = [ 1 3 5 2 ]  
p2 =  
1 3 5 2  
>> conv ( p1 , p2 )  
ans =  
1 1 0 -5 1 2
```

ÖRN:

```
y1=f(x)= x3 - 2 · x + 11  
y2=f(x)= x4 - 5 · x3 + 3 · x - 7  
y3=y1*y2  
>> y1= [ 1 0 -2 11]  
y1 =  
1 0 -2 11  
>> y2= [ 1 -5 3 -7]  
y2 =  
1 -5 3 -7  
>> y3= conv( y1, y2 )  
y3 =  
1 -5 1 14 -61 47 -77
```

* $\gg [q, r] = \text{deconv}(p1, p2)$: Fonksiyonu uzun bölme kullanarak $p1$ vektörünü $p2$ vektörüne böler. Bölüm q vektörü ile kalan ise r vektörü ile gösterilir ;

ÖRN:

```
y3=f(x)= x2 - 2 · x - 3
```

$$y_4=f(x)=x-3$$

```
>> y3=[ 1 -2 -3]
y3 =
     1     -2     -3
>> y4=[ 1 -3]
y4 =
     1     -3
>> [bolum,kalan]=deconv(y3,y4)
bolum =
     1     1
kalan =
     0     0     0
```

ÖRN :

```
>> p1=[ 1 1 0 -5 1 2]; p2=[ 1 -2 1];
>> deconv ( p1 , p2 )
ans =
     1     3     5     2
```

* >> q = polyint (p) : p polinomunun integralini bulur.

ÖRN :

```
>> p=[ 1 2 3];
>> int_p = polyint ( p )
int_p =
    0.3333    1.0000    3.0000     0
```

* >> k = polyder (p) : p polinomunun türevini bulur.

ÖRN :

```
turev_p = polyder ( int_p )
turev_p =
     1     2     3
```

* >> k = polyder (p1 , p2) : p1 ve p2 polinomlarının çarpımının türevini hesaplar.

ÖRN :

```
>> p1=[ 1 2]; p2=[ 1 3 5]; polyder ( p1 , p2 )
ans =
     3    10    11
```

* >> [q , d] = polyder (pay , payda) : Pay / payda polinomlarıyla verilen bölümün türevinin pay (q) ve payda (d) polinomlarını hesaplar.

ÖRN :

```
>> p1=[ 1 2]; p2=[ 1 3 5]; [ py , pyd ] = polyder ( p1 ,p2 )
py =
    -1    -4    -1
```

```
pyd =  
1 6 19 30 25
```

*** >> poly2str (p , ' x ')** : X karakterini değişken kabul ederek p polinomunu x değişkenine göre görüntüler.

ÖRN :

```
>> poly2str ( turev_p , ' x ' )  
ans =  
x ^2 + 2 x + 3
```

*** >> poly2sym (p , ' x ')** : X karakterini değişken kabul ederek p polinomunu x değişkenine göre görüntüler.

ÖRN :

```
>> poly2sym ( turev_p , ' x ' )  
ans =  
x^2 + 2*x + 3
```

*** >> tf (pay , payda)** : Pay ve payda polinomlarından $G(s) = P(s)/Q(s)$ transfer fonksiyonunu oluşturur.

ÖRN :

```
>> pay = [ 4 3 0 ] ; payda = [ 7 5 1 8 ] ;  
>> tf ( pay , payda )  
ans =  
4 s^2 + 3 s  
-----  
7 s^3 + 5 s^2 + s + 8  
Continuous-time transfer function.
```

*** >> [r , p , k] = residue (p1 , p2)** : Fonksiyonu pay (p1) ve payda (p2) polinomlarından oluşan iki polinom oranının kısmi kesirlere açılımının tam terimini , kalıntı ve kutupları hesaplar.

*** >> [p1 , p2] = residue (r , p , k)** : Fonksiyonu kısmi kesirlere açılımı pay (p1) ve payda (p2) polinomlarına dönüştürür.

ÖRN :

```
>> p1 = [ 1 2 3 ] ; p2 = [ 1 5 3 2 6 ] ;  
>> [ r , p , k ] = residue ( p1 , p2 )  
r =  
-0.1917 + 0.0000i  
0.1891 + 0.0000i  
0.0013 - 0.2175i  
0.0013 + 0.2175i  
p =  
-4.3418 + 0.0000i  
-1.3311 + 0.0000i  
0.3365 + 0.9617i  
0.3365 - 0.9617i  
k =  
[]
```

ÖRN :

```
>> [ pay , payda ] = residue ( r , p , k )
```

```

pay =
0.0000  1.0000  2.0000  3.0000
payda =
1.0000  5.0000  3.0000  2.0000  6.0000

```

* >> **a = polyfit (x , y , n)** : Fonksiyonu en küçük kareler mantığında $y = p (x)$ verilerini uyduran n dereceli $p (x)$ polinomunun katsayılarını hesaplar. a sonucu azalan kuvvetlerde polinom katsayılarını içeren $n + 1$ uzunluğunda bir satır vektörüdür.

ÖRN :

```

>> x = [ 1 2 3 4 5 6 ]; y = [ 18 27 125 216 343 ];
>> p = polyfit ( x , y , 2 )
p =
15.8571    -41.5143     24.8000
>> polyval ( p , 4 )
ans =
112.4571

```

MATLAB ' da Sayısal İntegral ve Türev alma

Sayısal integral :

* İntegral işleminin sayısal olarak elde edilmesinde yaygın olarak iki temel yöntem kullanılır;

Bunlar **trapezoidal (yamuk)** ve **simpson yöntemleri**'dir. MATLAB ' da belirli bir aralıkta sayısal integral alımı için **trapz** , **quad** ve **quad1** komutları kullanılır.

* Bu fonksiyonların kullanımları aşağıda örneklerle açıklamıştır ;

* >> **A = trapz (x , y)** : x vektörünün ilk değeri $(x (1))$ ile son değeri $(x (n))$ aralığında $y (x)$ fonksiyonunun integralini hesaplar.

$$A = \int_{x_1}^{x_n} f (x) . dx$$

ÖRN:

$[-1,1]$ aralığında

$$y = f(x) = e^x - x + 1$$

```

>> x = -1:0.01:1;
>> y = exp(x) - x + 1;
>> trapz (x,y)
ans =
4.3504

```

ÖRN :

```

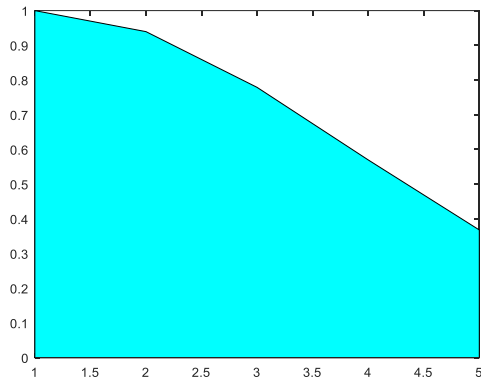
>> x = 0 : 0.25 : 1 ;           % ( x ( 1 ) = 0 , x ( n = 5 ) = 1 , integrasyon aralığı = 0.25
>> Alan = trapz ( x , exp ( -x.^2 ) )
Alan =
0.7430

```

NOT : İntegrasyon aralığı küçüldükçe , çözümün doğruluğu artacaktır.Eğer istenirse , hesaplanan alan grafiksel olarak da görüntülenebilir.

ÖRN :

```
>> area ( exp ( -x.^2 ) , 'facecolor ' , ' c ' )
```



*** >> quad (' fonk ' , a , b)** :Bu fonksiyon tekrarlamalı simpson yöntemine göre , fonk ile tanımlanan fonksiyonun $1e-3$ hata sınırları içerisinde a ' dan b ' ye kadar sayısal integralini hesaplar.

ÖRN :

```
>> f = inline ( ' exp ( -x.^2 ) ' )
f =
Inline function:
f(x) = exp ( -x.^2)
```

ÖRN :

```
x = 0 : 0.25 : 1 ;
Alan = quad ( f , 0 , 1 )
Alan =
0.7468
```

*** >> A = db1quad (' fonk ' , xmin , xmax , ymin , ymax)** : Bu fonksiyon iki değişkenli fonksiyonların iki katlı sayısal integralini hesaplar.

*** xmin ve xmax** : x değişkeninin sınırlarıdır.

*** ymin ve ymax** : y değişkeninin sınırlarıdır.

$$(A = \int_{x_{\min}}^{x_{\max}} \int_{y_{\min}}^{y_{\max}} f (x , y) dx dy)$$

ÖRN :

```
>> z = inline ( ' 3 * .^2 + 5 * y ' )
z =
Inline function:
z(y) = 3 * .^2 + 5 * y
```

ÖRN :

```
>> A=dblquad('3*x.^2+5*y',0,3,0,5)
A =
322.5000
```

* **A = triplequad (' fonk ' , xmin , xmax , ymin , ymax , zmin , zmax)** : Bu fonksiyon üç değişkenli fonksiyonların üç katlı sayısal integralini hesaplar.

* **xmin ve xmax** : x değişkeninin sınırları

* **ymin ve ymax** : y değişkeninin sınırları

* **zmin ve zmax** : z değişkeninin sınırları

$$(A = \int_{x_{\min}}^{x_{\max}} \int_{y_{\min}}^{y_{\max}} \int_{z_{\min}}^{z_{\max}} f (x , y , z) dx dy dz)$$

ÖRN :

```
>> f = inline ( ' y * sin ( x ) + z * cos ( z ) ' )
```

```
f =
```

Inline function:

```
f(x,y,z) = y * sin ( x ) + z * cos ( z )
```

ÖRN :

```
>> A = triplequad ( f , 0 , pi , 0 , 1 , -1 , 1 )
```

```
A =
```

```
2.0000
```

ÖRN :

```
>> A = triplequad ( ' y * sin ( x ) + z * cos ( z ) ' , 0 , pi , 0 , 1 , -1 , 1 )
```

```
A =
```

```
2.0000
```

TÜREV ALMA :

$$y = f(x) \quad \text{in türevi}$$

$$\frac{dy}{dx} = \frac{f(x + h) - f(x)}{(x + h) - (x)}$$

* MATLAB ' da **diff ()** fonksiyonu kullanılarak yaklaşık türev hesaplamaları yapılabilir.

$y = f (x)$ şeklinde tanımlanan bir fonksiyonun yaklaşık türevi , fonksiyon değeri farklarının değişken farklarına oranı esas alınarak **diff (y)./diff(x)** şeklinde hesaplanabilir.

NOT

: **diff ()** fonksiyonu ile x satır vektörünün eleman sayısının bir azaldığına dikkat edilmelidir.

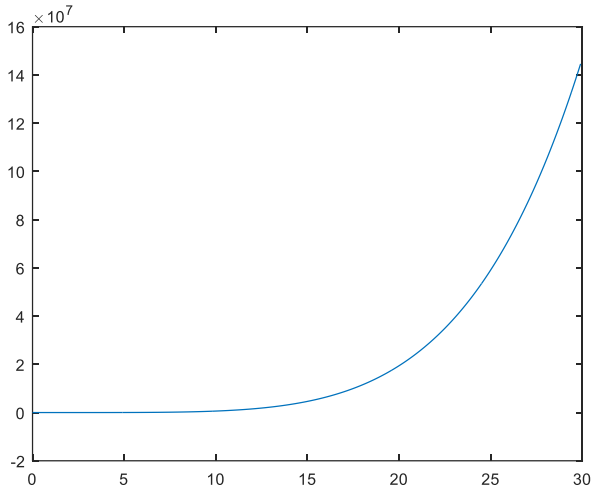
```
>> x=0:0.1:30;
```

```
>> y=x.^6-4*x.^3+4*x-8+sin(x);
```

```
>> turev_y=diff(y)./diff(x);
```

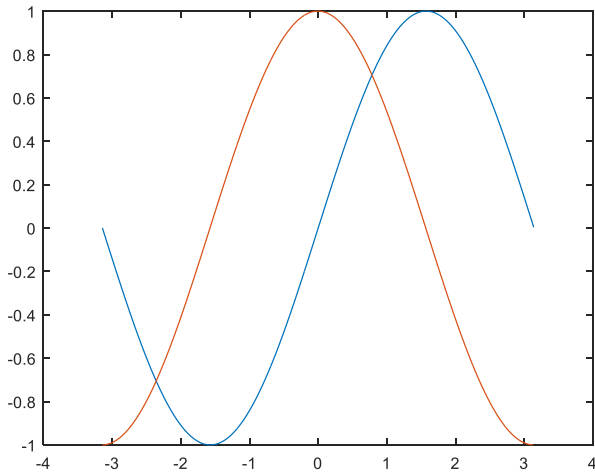
```
>> x = 0:0.1:29.9;
```

```
>> plot(x,turev_y)
```



ÖRN:

```
>> x=-pi:0.01:pi;
>> y=sin(x)
>> plot(x,y)
>> hold on
>> dy=diff(y)./diff(x);
>> x2=x(1:length(x)-1);
>> plot(x2,dy);
```



MATLAB ' da Diferansiyel Denklemlerin Çözümü

* MATLAB , her türlü diferansiyel denklemin sayısal çözümünü gerçekleştirebilecek çözüm fonksiyonları geliştirmiştir. MATLAB diferansiyel denklem çözüm fonksiyonları için *Ordinary Diderantial Equations* (**adi diferansiyel denklemler**) kelimelerinin baş harfleri olan **ODE** deyimini kullanmaktadır.

* ODE deyimi içerisine , analitik çözüm içeren veya içermeyen , sabit veya değişken katsayılı i birinci veya daha yüksek dereceden türev bulunduran bütün diferansiyel denklemler / denklem takımları girmektedir.

* Ayrıca sabit katsayılı doğrusal diferansiyel denklemlerin çözümü için , farklı dönüşüm yöntemleri ile bu denklemlerin çözümünü elde edebilen çeşitli çözüm fonksiyonları da bulunmaktadır.

* Verilen bir diferansiyel denklem takımı / takımlarının ODE çözücülerini ile çözümlerinin elde edebilmesi için;

1-) Diferansiyel denklemin tanımlandığı bir M-fonksiyon dosyasının eklenmesi gerekmektedir. ODE fonksiyonları hazırlanan bu diferansiyel denklem fonksiyon dosyalarını kullanarak çözüm yapabilmektedir.

2-) Diferansiyel denklemler ile sayısal çözümün gerçekleştirilebilmesi için problemin başlangıç koşullarının belirlenmesi gereklidir. Bu koşullar verilen bir t_0 zamanında y 'nin y_0 gibi bir değerine karşılık gelir.

* MATLAB ODE çözüm fonksiyonları sayısal integral alma yöntemlerini kullanır. Bu fonksiyonlar başlangıç koşulları ile bir t_0 başlangıç anından başlayıp , zaman aralığı boyunca adım adım ilerleyerek her bir zaman adımında bir çözüm hesaplar. Eğer çözüm yönteminin bir zaman adımında hesapladığı sonuç hata toleransını karşılarsa , çözüm başarılı sayılır ve sonraki adım için hesaplamalara geçilir. Aksi taktirde bu başarısız bir deneme kabul edilir ve fonksiyon adım boyutunu kısaltarak , hata toleransının altına düşene dek çözümü tekrarlanır.

ode45	Sıkı olmayan diferansiyel denklemler için orta derece çözüm yöntemi
Ode23	Sıkı olmayan diferansiyel denklemler için düşük derece çözüm yöntemi
Ode113	Sıkı olmayan diferansiyel denklemler için değişken derece çözüm yöntemi
Ode15s	Sıkı diferansiyel denklemler için değişken derece çözüm yöntemi
Ode23s	Sıkı diferansiyel denklemler için düşük derece çözüm yöntemi
Ode23t	Kısmen sıkı diferansiyel denklemler için yamuk kurallı çözüm yöntemi
Ode23tb	Sıkı olmayan diferansiyel denklemler için düşük derece çözüm yöntemi

Diferansiyel Denklemlerin ODE Fonksiyonları İle Çözdürülmesi Ve Çözümün Değerlendirilmesi

* Başlangıç koşulları tanımlanmış herhangi bir diferansiyel denklem takımının ODE çözüm fonksiyonları ile çözümünün ve değerlendirilmesinin nasıl yapılacağı basit bir örnek ile adım adım açıklanacaktır.

ÖRN :

$y''' - 3y'' - y'y = 0$ ve başlangıç koşulları $y(0) = 0$, $y'(0) = 1$, $y''(0) = -1$ olarak tanımlanmış bir diferansiyel denklemdir.

1-) Birinci dereceden diferansiyel denklem takımına dönüştürme ;

* Çözümü gerçekleştirilecek diferansiyel denklem 3. mertebeden olduğu için , değişken değiştirme yöntemi ile bu denklem birinci mertebeden denklem takımlarına dönüştürülür.

* Değişken değişimi için $y_1 = y$ değişken değişiminden başlanarak ;

$$y'_1 = y_2$$

$$y'_2 = y_3$$

$$y'_3 = 3y_3 + y_2y_1$$

* Üçüncü dereceden diferansiyel denklem üç tane birinci dereceden diferansiyel denklem takımına dönüştürülür.

Bu durumda diferansiyel denklem takımının başlangıç koşulları da aşağıdaki gibi elde edilir.

$$y_1(0) = 0 ;$$

$$y_2(0) = 1 ;$$

$$y_3(0) = -1 ;$$

Burada ;

$$Y' = F(T, Y), Y(0) = Y_0 \text{ ve } Y = [y_1, y_2, y_3]$$

2-) İkinci dereceden diferansiyel denklem takımı için M-fonksiyon dosyasının oluşturulması ;

* Elde edilen birinci dereceden denklem takımı için M-fonksiyon dosyası oluşturulur. M- fonksiyon dosyası , t ve y şeklinde iki giriş parametresi kabul eden ve çıkış olarak bir sütun vektörü geri döndürecek şekilde kodlanır.

ÖRN :

```
function dy = F ( t , y )  
dy = [ y ( 2 ) ; y ( 3 ) ; 3*y ( 3 ) + y ( 2 ) * y ( 1 ) ] ;
```

* Fonksiyon adı ' F ' olarak belirlendiği için M dosyası da ' F ' adıyla kaydedilir. Burada function ile başlayan fonksiyon dosyasının en azından iki adet giriş parametresi bulunmalıdır. (Bu parametrelerin adının t ve y olması zorunlu değildir)

* dy ise bir sütun vektörü şeklinde tanımlanmalıdır.(Benzer şekilde adının dy olması zorunlu değildir.)

3-) Hazırlanan fonksiyon dosyasının çalıştırılması ve çözümün elde edilmesi ;

* Diferansiyel denklemin M- fonksiyon dosyası oluşturulduktan sonra , ODE çözüm fonksiyonlarından birisi seçilerek denklem çözümü elde edilir.

ODE çözüm fonksiyonlarının genel kullanımı ;

[T , Y] = odexx (' Fonksiyon Adı ' , tspan , y0) % ODEXX('fonksiyonadi',0,1,[y(0)])

Fonksiyon Adı : Diferansiyel denklemin oluşturulduğu M-fonksiyon dosyasının adı
tspan : [tbaşlangıç , tson] şeklinde tanımlanan integral zaman aralığı
y0 : Başlangıç (t₀) koşullarını içeren sütun vektörüdür.

* Ele alınan örnek problemin ode45 fonksiyonu ile [0 1] zaman aralığındaki çözümünü bulmak için ,
fonksiyon çağırma düzeninin ,
genel yazımı ;

[T , Y] = ode45 (' F ' , [0 1] , [0 ; 1 ; -1]) ;

4-) Çözüm Sonuçlarının Değerlendirilmesi ;

Çözüm fonksiyonu ile elde edilen sonuçlar , çıkış parametreleri olarak tanımlanan T ve Y adlı iki değişkene atanır.

* **T değişkeni** : sütun vektörü olarak çözümün zaman noktası değerlerini saklar. Bu zaman değerleri , integral hesabında kullanılan değerler değildir. Çözümün zamana göre değişimini görebilmemiz için oluşturulan uygun zaman değerleridir.

* **Y değişkeni** : T kadar satır sayısına ve M-fonksiyon dosyasında tanımlanan birinci dereceden diferansiyel denklem sayısı kadar sütuna sahip bir matristir.

* Bu örnekte T değişkeni 57x1 boyutunda ve üç tane birinci dereceden diferansiyel denklem olduğu için Y 'nin boyutu 57x3'tür.

* Y matrisinin ;

Birinci sütununda $Y(:, 1)$ şeklinde T zaman adımlarındaki çözümün kendisi ($y_1 = y$ olarak tanımlanmıştı) ,
İkinci sütunu $Y(:, 2)$ biçiminde T zaman adımlarındaki çözümün birinci türevi ($y'_1 = y_2$ olarak tanımlanmıştı) ve
 $Y(:, 3)$ biçiminde T zaman adımlarındaki çözümün ikinci türevinin ($y'_2 = y_3$ olarak tanımlanmıştı) çözümü bulunur.

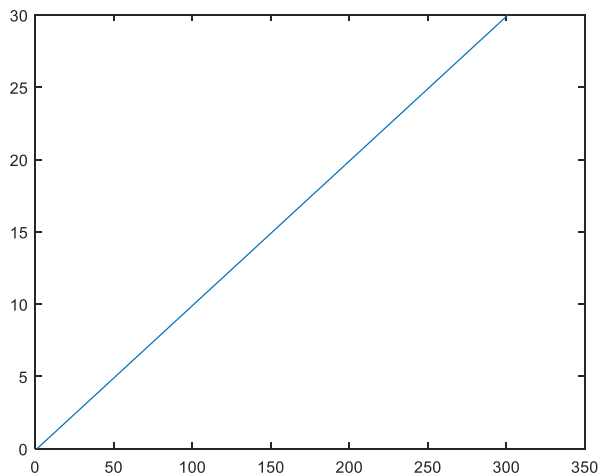
* Çözüm sonucu grafiksel olarak aşağıdaki gibi görüntülenebilir.

```
>> plot ( T , Y ( : , 1 ) )
```

Benzer şekilde y' ve y'' de grafiksel olarak görüntülenebilir.

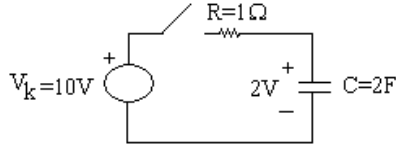
ÖRN :

```
function dy=F1(t,y)
dy1=y(2);
dy2=y(3);
dy3=3*y(3)+y(2)*y(1);
dy=[dy1;dy2;dy3];
>> y=ode23('F1',[0 2],[0;1;-2])
y =
    0
 0.0001
 0.0005
 0.0025
 0.0125
 0.0490
 0.1040
 0.1696
.....
```



ÖRN:

Şekildeki devrede $t=0$ anında anahtar kapanıyor. Kapasitör geriliminin 0-10sn aralığındaki değişimini bulunuz ve çizdiriniz.



$$\frac{dV_c}{dt} = \frac{V_k - V_c}{R \cdot C}$$

function dVc=gerilim(t,Vc) % C kapasitörünün % geriliminin değişimi

R=1; C=2; Vk=10;

dVc=(Vk-Vc)/(R*C);

>> Vc0=2;

>> tara=[0 10];

>> ode45('gerilim',tara,Vc0);

>> [t,x]=ode45('gerilim',tara,Vc0);

>> plot(t,x)

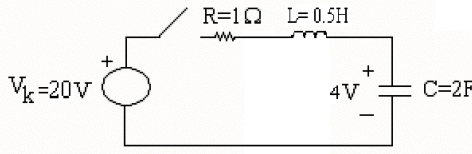
>> title('Kapasitör geriliminin değişimi')

>> xlabel('Zaman')

>> ylabel('Gerilim')

ÖRN:

Şekildeki devrede $t=0$ anında anahtar kapanıyor. Kapasitör geriliminin ve bobin akımının 0-15sn aralığındaki değişimini bulunuz ve çizdiriniz.



$$\frac{dI_L}{dt} = \frac{1}{L} (V_k - V_c - R \cdot I_L)$$

$$\frac{dV_c}{dt} = \frac{1}{C} \cdot I_L$$

$$\frac{dI_L}{dt} = dx(1) \quad , \quad \frac{dV_c}{dt} = dx(2)$$

$$I_L = x(1) \quad , \quad V_c = x(2)$$

function dx=gerilim(t,x) % C kapasitörünün % geriliminin ve L bobininin % akımının değişimi

R=1; C=2; L=0.5; Vk=20;

$$\frac{dI_L}{dt} = \frac{1}{L} (V_k - V_c - R \cdot I_L)$$

dx(1)=(1/L)*(Vk-x(2)-R*x(1));

$$\frac{dV_c}{dt} = \frac{1}{C} \cdot I_L$$

dx(2)=(1/C)*x(1);

dx=dx';

>> x0=[0 4];

>> tara=[0 15];

>> ode45('rlc',tara,x0);

>> [t,x]=ode45('rlc',tara,x0);

>> plot(t,x)

MATLAB 'DA DOĞRUSAL SİSTEMLERİN ANALİZİ

MATLAB ,Control system Toolbox içerisinde doğrusal zamanla değişmeyen parametrelili (LTI-Linear Time Invariant) sistemleri araştırmak ve analiz etmek amacıyla çok kapsamlı araçlar sunmaktadır.

* Burada "**LTI**" sistemlerden kasıt ;

Dinamik davranışları veya sistem denklemleri sabit katsayılı doğrusal diferansiyel denklemler ile ifade edilen doğrusal sistemlerdir.

* LTI sistemlerinde , sistemlerin matematiksel modellerini tanımlamak amacıyla değişik yöntemler kullanılır.

Bu yöntemlerin en önemlileri ;

1-) Diferansiyel denklemler ile gösterim (zaman alanı)

2-) Durum uzayı veya durum denklemleri ile gösterim (zaman alanı)

3-) Transfer fonksiyonu ile gösterim (Laplace dönüşümü veya karmaşık s-alanı) ,

4-) z-alanı (ayırık zaman alanı) ile gösterimdir.

Diferansiyel denklem gösterimi :

* Herhangi bir sabit katsayılı doğrusal sistemin diferansiyel denklemi ;

$$a_n \frac{d^n y}{dt^n} + a_{n-1} \frac{d^{n-1} y}{dt^{n-1}} + \dots + a_1 \frac{dy}{dt} + a_0 y = b_m \frac{d^m x}{dt^m} + b_{m-1} \frac{d^{m-1} x}{dt^{m-1}} + \dots + b_1 \frac{dx}{dt} + b_0 x$$

şeklinde dir. Burada ;

a ve b : Sabit sayılar ,

y = y (t) : Sistem çıkış değişkeni

x = x (t) : Sistemin giriş ve uyarım değişkeni

* LTI diferansiyel denklemin çözümü ODE fonksiyonları ile gerçekleştirilir.

Durum Denklemleri Gösterimi :

* Özellikle çok giriş çok çıkışlı sistemlerin matematiksel denklemleri , birinci dereceden diferansiyel denklemlere indirgenmiş halde gösterilir. Birinci dereceye indirgenmiş diferansiyel denklem tanımları durum denklemleri olarak adlandırılır.

Durum denklemi ve çıkış denklemi matris şeklinde aşağıdaki gibi gösterilir ;

$$\frac{d}{dt} \mathbf{X} (t) = \mathbf{A} \mathbf{X} (t) + \mathbf{B} \mathbf{u} (t)$$

$$\mathbf{y}(t) = \mathbf{C} (t) + \mathbf{D} \mathbf{u}(t)$$

X : Durum vektörü (n-elemanlı sütun vektörü)

u : Denetim (giriş) vektörü (r-elemanlı sütun vektörü)

y : Çıkış vektörü (m-eleman satır vektörü)

A : nxn elemanlı matris

B : nxr elemanlı matris

C : mxn elemanlı matris

D : mxr elemanlı matris

ÖRN :

```


$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -5 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 3 \end{bmatrix} u$$


$$y = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + 0u \quad \% y = x_1 * 0 + 1 * x_2 + 0 * u$$

>> s_dd = ss ([ 0 1 ; -5 -2 ], [ 0 : 3 ], [ 0 1 ], 0)
a =
    x1 x2
x1  0  1
x2 -5 -2
b =
    u1
x1  0
x2  3
c =
    x1 x2
y1  0  1
d =
    x1 x2
y1  0

```

ÖRN :

```


$$T = \frac{3s+5}{s^2+2s+7}$$

yazılımı ;
pay : [ 3 5 ] , payda : [ 1 2 7 ]
t= tf ( pay payda )      % tf = transfer fonksiyonu

```

Transfer Fonksiyonu Gösterimi :

Transfer fonksiyonları doğrusal sistemlerin modellenmesinde çok yaygın olarak kullanılırlar.

* Bir sistemin transfer fonksiyonu ;

Sistemin bütün başlangıç koşullarının sıfır kabul ile , çıkış fonksiyonu Laplace dönüşümünün giriş fonksiyonu Laplace dönüşümüne oranı olarak tanımlanır. Buna göre transfer fonksiyonları bir karmaşık s-alanı gösterimidir.

* Transfer fonksiyonları kendi aralarında aşağıda tanımlanan değişik gösterimlere sahiptir.

Polinom Gösterimi :

* Transfer fonksiyonunun pay ve paydası aazalan polinom şeklinde gösterilir.

$$G(s) = \frac{Y(s)}{X(s)} = \frac{b_ms^m + b_{m-1}s^{m-1} + \dots + b_1s + b_0}{a_ns^n + a_{n-1}s^{n-1} + \dots + a_1s + a_0}$$

$$G(s) = \frac{s}{s^2 + 2s + 10}$$

ÖRN :

```
>> g = tf ([ 1 0 ], [ 1 2 10 ])
```

```
g =
```

```
    s
```

```
-----
```

```
s^2 + 2 s + 10
```

Continuous-time transfer function.

Sıfır Kutup - Kazanç Gösterimi :

* Transfer fonksiyonu , sıfır , kutup ve kazanç katsayısının çarpanları şeklinde gösterilir.

$$G(s) = \frac{Y(s)}{X(s)} = \frac{k(s-z_1)(s-z_2)....(s-z_m)}{(s-p_1)(s-p_2)....(s-p_n)}$$
$$G(s) = -2 \frac{s}{(s-2)(s-1+j)(s-1-j)}$$

ÖRN :

>> g = zpk(0, [-2 -1 +j -1 -j], -2)

Zero / pole * gain :

g =

-2 s

(s+(3-1i))(s+(1+1i))

Continuous-time zero/pole/gain model.

Kısmi Kesirli Gösterim :

* Transfer fonksiyonu basit kesirler halinde gösterilir.

$$G(s) = \frac{Y(s)}{X(s)} = \frac{r_1}{(s-p_1)} + \frac{r_2}{(s-p_2)} + \dots + \frac{r_n}{(s-p_n)} + k(s)$$
$$G(s) = \frac{s}{s^2+2s+10}$$

ÖRN :

>> [r, p, k] = residue([1 0], [1 2 10]) %Burada; r= pay ; p=payda ; k=kalan

r =

0.5000 + 0.1667i

0.5000 - 0.1667i

p =

-1.0000 + 3.0000i

-1.0000 - 3.0000i

k =

[]

ÖlÜ Zaman Sabitine Sahip Sistemlerin Modellenmesi :

MATLAB ile sürekli zamanlı sistemlerde ölü zaman gecikmesi oluşturulabilmektedir. Ayrık zaman sistemlerinde ise sıfırıncı ve birinci dereceden tutucular ölü zamanlı sistemleri desteklemektedir.

$$G(s) = e^{-0.25s} \frac{10}{s^2 + 3s + 10}$$

ÖRN :

>> g=tf(10,[1 3 10],'inputdelay',0.25)

g =

10

exp(-0.25*s) * -----

s^2 + 3 s + 10

Continuous-time transfer function.

Not

:Varolan bir sisteme de ölü zaman eklenebilir.

ÖRN :

```
>> g = tf ( 10 , [ 1 3 10 ] );
>> set(g,'inputdelay',0.25)
>> g
g =
          10
exp(-0.25*s) * -----
          s^2 + 3 s + 10
```

Continuous-time transfer function.

MEVCUT BİR SİSTEMİN GÖSTERİMİNİN DEĞİŞTİRİLMESİ

* MATLAB ile LTI sistem modellerinin dönüşümü gerçekleştirilebilir. Ancak modeller arası dönüşümlerde dikkatli olmak gerekir. Özellikle yüksek dereceli sistemlerin dönüşüm sonuçlarının güvenilirliği tartışmalıdır.

```
sys = tf ( sys )      % sys adlı sistemi transfer fonksiyonu gösterimine dönüştürür.
sys = zpk ( sys )    % sys adlı sistemi sıfır - kutup kazanç gösterimine dönüştürür.
sys = tf ( sys )      % sys adlı sistemi durum denklemi gösterimine dönüştürür.
```

ÖRN :

$$\begin{bmatrix} \dot{X}_1 \\ \dot{X}_2 \\ \dot{X}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -6 & -11 & -6 \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 6 \end{bmatrix} U$$

$$y = [1 \ 0 \ 0] \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$$

```
>> A = [ 0 1 0 ; 0 0 1 ; -6 -11 -6 ] ; B = [ 0 ; 0 ; 6 ] ; C = [ 1 0 0 ] ; D = [ 0 ] ;
```

```
>> sys = ss ( A , B , C , D )
```

```
sys =
```

```
a =
```

```
      x1  x2  x3
```

```
x1  0  1  0
```

```
x2  0  0  1
```

```
x3 -6 -11 -6
```

```
b =
```

```
      u1
```

```
x1  0
```

```
x2  0
```

```
x3  6
```

```
c =
```

```
      x1  x2  x3
```

```
y1  1  0  0
```

```
d =
```

```
      u1
```

```
y1  0
```

ÖRN :

```
>> sys = zpk ( sys )
```

```
sys =
```

```
      6
```

```
-----
```

```
(s+3) (s+2) (s+1)
```

Continuous-time zero/pole/gain model.

ÖRN :

```
>> sys = tf ( sys )
```

```
sys =
```

```
6
```

```
-----  
s^3 + 6 s^2 + 11 s + 6
```

Continuous-time transfer function.

ÖRN :

```
>> sys = ss ( sys )
```

```
sys =
```

```
a =
```

```
      x1  x2  x3
```

```
x1  -6 -2.75 -1.5
```

```
x2   4   0   0
```

```
x3   0   1   0
```

```
b =
```

```
      u1
```

```
x1  1
```

```
x2  0
```

```
x3  0
```

```
c =
```

```
      x1  x2  x3
```

```
y1  0  0  1.5
```

```
d =
```

```
      u1
```

```
y1  0
```

Continuous-time state-space model.

LTI SİSTEMLERİN ZAMAN ALANI ANALİZİ

Step Fonksiyonu :

Tanımlanan bir LTI sisteminin sıfır başlangıç koşullarında birim basamak yanıtını bularak çizimi yapar. Bu fonksiyon ; sayısal hesaplama yöntemini kullanarak sürekli zaman sistemini önce kendisine denk bir ayırık zaman sistemine dönüştürdükten sonra gerekli işlemleri yaparak çözümü gerçekleştirir.

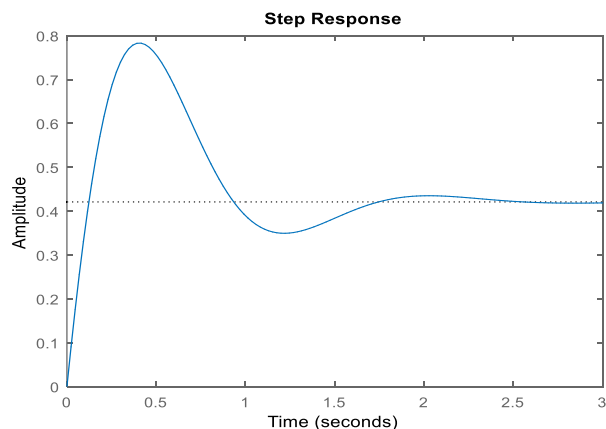
Step fonksiyonunun kullanımı ;

* >> **step (sys)** : Burada sys ; tf , ss veya zpk ile oluşturulan LTI modelidir.

ÖRN :

```
>> sys = tf ( 4 * [ 1 2 ] , [ 1 4 19 ] ) ;
```

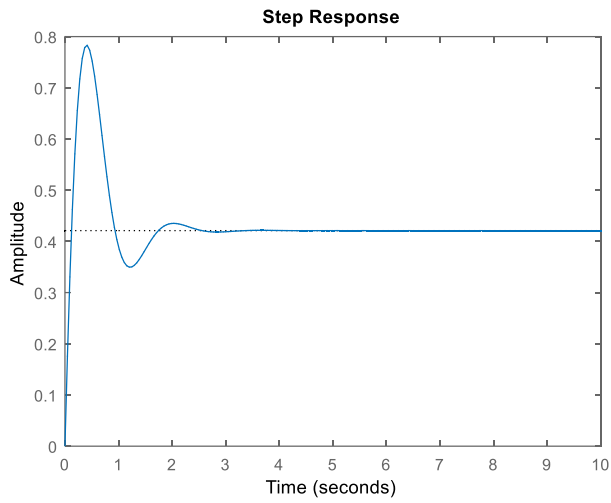
```
>> step ( sys )
```



* >> **step (sys , tson)** : Burada tson çözümün sonlandırılacağı andır.

ÖRN :

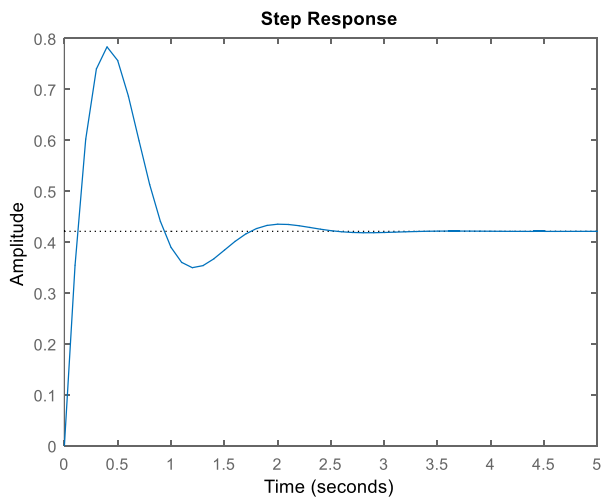
>> step (sys , 10)



* >> **Step (sys , t)** : Burada t sys ' nin çözümü için kullanıcının tanımladığı zaman vektörüdür.

ÖRN :

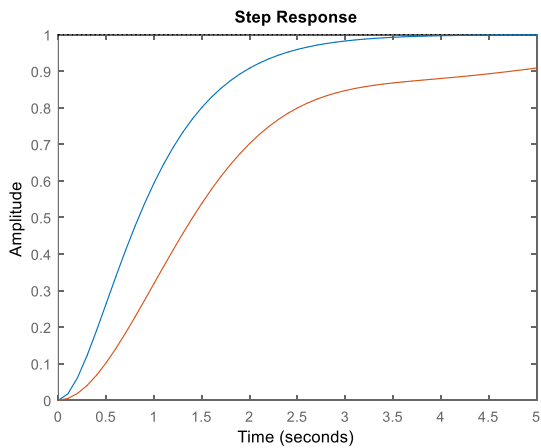
>> step (sys , [0 : 0.1 : 5])



* >> **Step (sys1 , sys2 , ... , t)** : Birden fazla sistemin aynı anda aynı zaman vektörü ile çözümünü üretir.

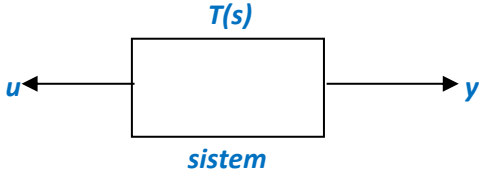
ÖRN :

>> sys1 = tf (4 , [1 4 4]) ; sys2 = tf ([1 1] , [1 2 3 1]) ;
>> step (sys1 , sys2 , [0 : 0.1 : 5])



* >> [y, t] = step (sys) : Üretilen sonucu y vektörüne zamanı ise t vektörüne atar.
 >> [y t] = step (sys)

* >> square : Kare dalga

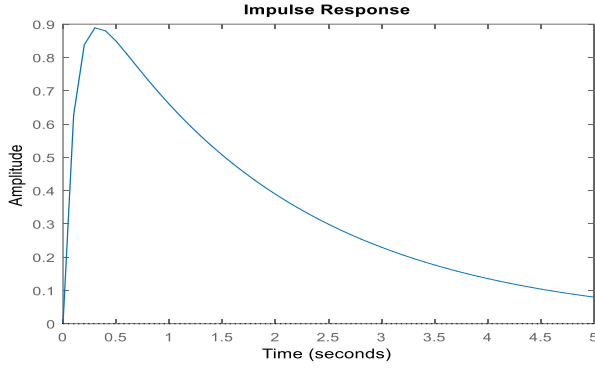


impulse fonksiyonu :

Tanımlanan LTI sisteminin birim ani darbe yanıtını üreterek , elde edilen çözümün grafiğini çizdirir. Kullanımı tamamen step fonksiyonu ile aynıdır. Yukarıda step fonksiyonu için verilen söz dizimlerinin tamamı impulse fonksiyonu için de geçerlidir.

ÖRN :

```
>> sys = tf( 10 , [ 1 10 5 ] ) ; t = 0 : 0.1 : 5 ;
>> impulse ( sys , t )
```



Isim fonksiyonu :

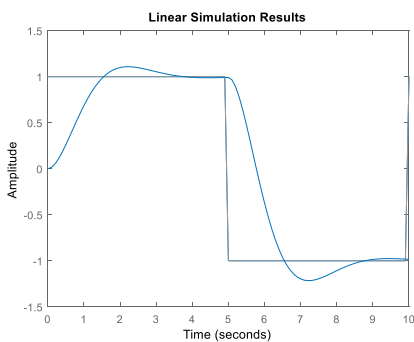
Tanımlanan bir LTI sistemi ve bu sisteme tanımlanan bir girişin uygulamasıyla elde edilen sonucun grafiğini çizdirir. Söz dizimi aşağıdaki gibi tanımlanır.

```
* >> Isim ( sys , u , t ) :
sys : Matematiksel model ;
u : Giriş ;
t : Zaman aralığı
```

için sıfır başlangıç koşullarında çözümünü gerçekleştirerek sonucun grafiğini çizdirir.

ÖRN:

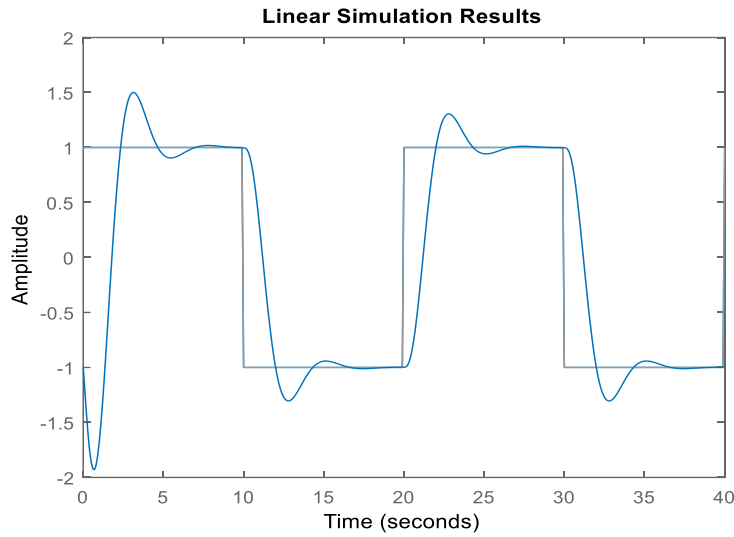
```
>> sys = tf( 3 , [ 1 2 3 ] ) ; t = 0 : 0.1 : 10 ;
>> u = square ( 2 * pi * 0.1 * t ) ; plot ( t , u ) ;
>> hold on , lsim ( sys , u , t )
```



*** >> lsim (sys , u , t , x0) :**Yukarıdaki komuta ek olarak tanımlanan x0 başlangıç koşulları altında çözümü gerçekleştirir.Başlangıç koşullarının değerlendirilebilmesi için , sys sisteminin ss ile (durum denklemi) tanımlanması gereklidir. Eğer sistem tf yada zpk ile tanımlanmış ise x0 değerlendirilmez.

ÖRN :

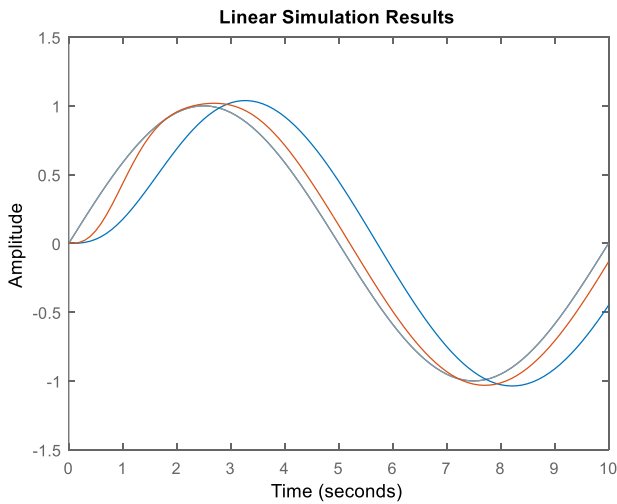
```
>> sist = ss ( [ 0 1 0 ; 0 0 1 ; -6 -6 -4 ] , [ 0 ; 0 ; 6 ] , [ 1 0 0 ] , [ 0 ] );
>> t = 0 : 0.1 : 40 ; u = square ( 2 * pi * 0.05 * t ) ; plot ( t , u ) ;
>> hold on , lsim ( sist , u , t , [ -1 -2 -3 ] )
```



*** >> lsim (sys1 , sys2 , ... , u , t , x0) :**Birden fazla sistemin çözümünü aynı t-zaman vektörü için gerçekleştirerek sonucun grafiğini aynı eksen üzerinde çizdirir.

ÖRN :

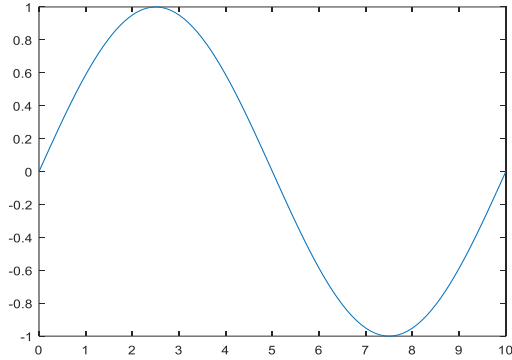
```
>> sys1 = tf ( 3 , [ 1 2 3 ] ) ; sys2 = tf ( 10 , [ 1 2 10 ] ) ;
>> t = 0 : 0.1 : 10 ; u = sin ( 2 * pi * t * 0.1 ) ; plot ( t , u ) ;
>> hold on , lsim ( sys1 , sys2 , u , t )
```



* >> [ys,ts] : lsim (sys , u , t , x0) :
 Sys sisteminin çözümünü yaparak ;
 çözüm sonucunu "ys",
 zamanı ise "ts" vektörüne atar.

ÖRN :

```
>> u=sin(2*pi*t*0.1); plot(t,u);
>> [ y,t ] = lsim ( sist , u , t , [ -1 -2 0 ]
```



```
y =
-1.0000
-1.1972
.....
t =
0
0.1000
.....
```

Sayı Sistemleri Arasında Dönüşüm Sağlayan Komutlar

* >> bin2dec : İkilik kodla yazılmış bir sayıyı (binary) onluk (decimal) koda dönüştürür.

Kullanımı:

```
bin2dec(' Binary sayı ')
```

ÖRN :

```
>> bin2dec('11001')
```

* >> dec2bin : Onluk (decimal) kodla yazılmış bir sayıyı ikilik (binary) kod dizisine dönüştürür.

Kullanımı:

```
>> dec2bin( ' Onluk sayı ' )
```

ÖRN :

```
>> dec2bin(25)
```

* >> dec2hex : Onluk tabandaki bir sayıyı 16 lık tabanda bir diziye (hexadecimal) dönüştürür.

Kullanımı:

```
>> dec2hex('onluk sayı')
```

ÖRN:

```
>> dec2hex(34)
```

* >> hex2dec : 16 lık tabanda bir diziye onluk tabanda bir sayıya dönüştürür.

ÖRN:

```
>> hex2dec('AB12')
```

MATLAB' DA KONTROL KOMUTLARI PAKETİ(TOOLBOX)

* Bir sistemin matematiksel ifadesi

1-)Durum uzay gösterimi

2-)Transfer fonksiyonu gösterimi

MATLAB ' da bir sistemin tanımlanması ;

1-)Durum uzay gösterimi ;

$$\dot{X} = A \cdot X + B \cdot u$$

$$y = C \cdot X + D \cdot u$$

Sisteme ait olan A,B,C,D matrisleri tanımlanır.

* >> ss(A,B,C,D) : Durum uzay modeli girişinde kullanılır.

ÖRN :

$$\dot{X} = \begin{bmatrix} 0 & 1 \\ -3 & -2 \end{bmatrix} \cdot X + \begin{bmatrix} -1 \\ 0 \end{bmatrix} \cdot u$$

$$Y = [0 \quad 1] \cdot X + D \cdot u$$

>> A=[0 1; -3 -2]

>> B=[-1; 0]

>> C=[0 1]

>> D=0

>> sistem1=ss(A,B,C,D)

2) Transfer fonksiyonu gösterimi;

$$\frac{y}{u}(s) = \frac{a_1 \cdot s^n + a_2 \cdot s^{n-1} + \dots + a_n}{b_1 \cdot s^m + b_2 \cdot s^{m-1} + \dots + b_m}$$

pay=[a₁ a₂ ... a_n]

payda=[b₁ b₂ b_m]

ÖRN :

$$\frac{y}{u}(s) = \frac{s + 2}{s^2 - s - 12}$$

>> pay= [1 2]

>> payda= [1 -1 -12]

>> sistem2=tf(pay,payda)

Sistem Gösterimleri Arasında Dönüşüm

* Durum uzay modelinden transfer fonksiyonuna dönüşüm ;

* >> ss2tf (A,B,C,D) :Durum uzay modelinden transfer fonksiyonu gösterimine geçişi sağlar.
(from state space to transfer function)

ÖRN :

>> ss2tf(A,B,C,D)

* Transfer fonksiyonu gösteriminden durum uzay modeline dönüşüm ;

* >> tf2ss (pay,payda) : transfer fonksiyonu gösteriminden durum uzay modelinde dönüşümü sağlar.
(from transfer function to state space)

ÖRN :

>>tf2ss(pay,payda)

Kutup – Sıfır Formuna Dönüşüm:

* >> **zpk(sistem)** : Sistemin kutup ve sıfırlarının düzenlenmiş halini elde eder. Sistem MATLAB’ da ister transfer fonksiyonu formunda ister durum uzay formunda olsun kutup sıfır gösterimi elde edilir.

ÖRN :

>> zpk(sistem1)

>> zpk(sistem2)

* Sistemin kutuplarının bulunması ;

* >> **pole** : Sistemin kutuplarının bulunmasında kullanılır.

ÖRN :

>> pole(sistem1)

>> pole(sistem2)

* Sistemin sıfırlarının bulunması ;

* >> **zero** : Sistemin sıfırlarının bulunmasında kullanılır.

ÖRN :

>> zero(sistem1)

>> zero(sistem2)

Kutup-Sıfır Dağılımının Gösterimi:

* >> **pzmap** : İlgilenilen sisteme ait kutup ve sıfırların gösteriminde kullanılır.

ÖRN :

>> pzmap(sistem1)

Sönüm oranının ve Doğal Frekansın Bulunması :

* >> **damp** : Tanımlanmış sistemin özdeğerlerini, doğal frekansını ve sönümünü bulmada kullanılan komuttur.

ÖRN :

>> damp(sistem1)

>> damp(sistem2)

Dc Kazanç Hesaplanması:

* >> **Dcgain** : Sistemin DC kazancını hesaplar.

($s \rightarrow j\omega$, Dc de $\omega \rightarrow 0$)

ÖRN :

>> dcgain(sistem1)

Sürekli Zaman Sisteminden Ayrık Zamanlı Sisteme Geçiş :

* >> **C2d** : Sürekli zaman sisteminden ayrık zamanlı sisteme geçişi sağlar.

SYSD=C2D(SYSC,TS,METHOD)

TS : Örnekleme periyodu

METHOD

'ZOH' : 0. dereceden tutucu

'FOH' : 1. dereceden tutucu

'TUSTIN' : Bilineer dönüşüm

* >> d2c : Ayrık zamanlı sistemden sürekli zaman sistemine dönüşümü sağlar.

ÖRN :

>> SYSC = D2C(SYSD,METHOD)

METHOD:

'ZOH' : 0. dereceden tutucu

'FOH' : 1. dereceden tutucu

'TUSTIN' : Bilineer dönüşüm

Kontrol Sistemlerine Yönelik Analizler :

* Kök Yer Eğrisi çizimi ;

ÖRN :

>> rlocus(sistem)

>> rlocfind(sistem)

* Bode Eğrisi Çizimi ;

ÖRN :

>> bode(sistem)

>> margin(sistem)

UYGULAMALAR

1-)Silindirin yüzey alanı (uçları kapalı silindir için)

>> r=5;

>> h=10;

>> silindir=2*pi*r^2+2*pi*r*h % veya >> surface_area=2*pi*radius*(radius+height);

silindir =

471.2389

2-)Aşağıdaki matematik işlemlerini gerçekleştirecek matematiksel işlemleri yapınız ?

$$f = \frac{\log(ax^2+bx+c) - \sin(ax^2+bx+c)}{4\pi x^2 + \cos(x-2)*(ax^2+bx+c)} \quad x=9; a=1; b=3; c=5;$$

>> x=9

x =

9

>> b=3

b =

3

>> a=1

a =

1

>> c=5

c =

5

>> f=(log(a*x^2+b*x+c)-sin(a*x^2+b*x+c))/(4*pi*x^2+cos(x-2)*(a*x^2+b*x+c))

f =

0.0044

poly=a*x^2+b*x+c

poly =

113

>> denom=4*pi*x^2+cos(x-2)*poly

denom =

1.1031e+03

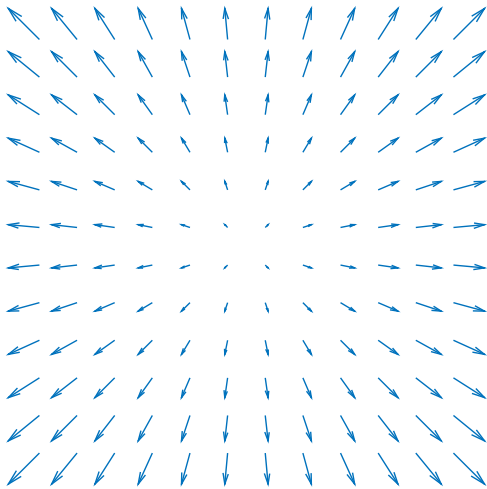
>> f=(log(poly)-sin(poly))/denom

f =

0.0044

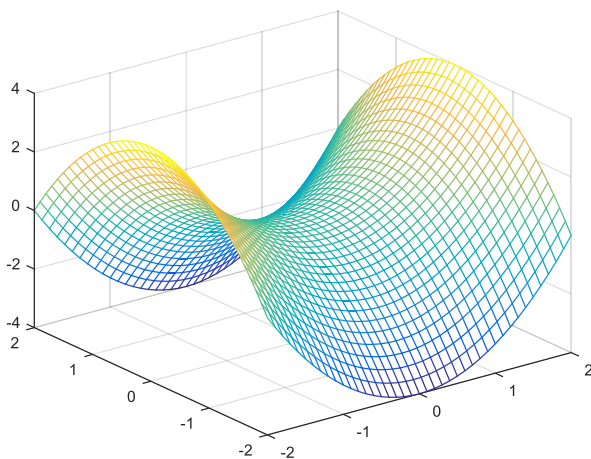
3-)

```
>> [x,y] = meshgrid(-1.1:2:1.1,-1.1:2:1.1)
>> quiver(x,y);axis equal; axis off
```



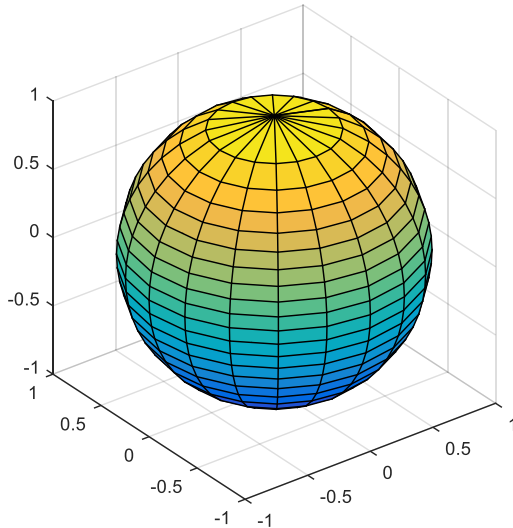
4-)

```
>> [x,y]=meshgrid(-2:.1:2,-2:.1:2);
>> z=x.^2-y.^2;
>> mesh(x,y,z)
```



5-)

```
>> [theta,z]=meshgrid((0:0.1:2)*pi,(-1:0.1:1));
>> x=sqrt(1-z.^2).*cos(theta);
>> y=sqrt(1-z.^2).*sin(theta);
>> surf(x,y,z);
>> axis square
```



6-) $\int \frac{1}{(x^2-25)} dx$ belirsiz integralini hesaplayınız.

```
>> syms x
>> int(1/(x^2-25))
ans=
1/10*log(x-5)-1/10*log(x+5)
```

7-)

```
>> syms a positive;
>> syms x
>> f=exp(-a*x^2);
>> sonuc=int(f,x,-inf,inf)
sonuc =
pi^(1/2)/a^(1/2)
>> pretty(sonuc)
sqrt(pi)
-----
sqrt(a)
```

8-) Aşağıda verilen matematiksel ifadelerin sonuçlarını $x=5$ için veren MATLAB komutlarını yazınız.

a-) $2x^2 + 3x - 4$

```
>> 2*x^2+3*x-4
```

ans=

61

b-) $\frac{x^3+1}{2x^5-2x^2+3x}$

```
>> (x^3+1)/(2*x^5-2*x^2+3*x)
```

ans=

0.0203

c-) $\lim_{10} (x - 3x)^2 + 4\sqrt{x^3}$

```
>> log10(x-3*x)^2+4*sqrt(x^3)
```

ans =

43.8598 + 2.7288i

9-) Aşağıdaki ifadeleri MATLAB formunda yazınız.

a-) $ax^n - c \cdot 10^{-k}$ = >> $a*x^n - c*10^{-k}$

b-) $\sqrt{(uv)^{2-s}} - t$ = >> $\text{sqrt}((u*v)^(2-s))-t$

c-) $(m/3)2n(k-3r)$ = >> $(m/3)*2*n*(k-3*r)$

d-) $(ab^g + 3)/c^{-1/h}$ = >> $(a*b^g+3)/(c^(-1/h))$

10-) Aşağıdaki ifadeleri MATLAB formatında yazarak sonuçlarını bulunuz.

a-) $\gg 10\log 100 = 10*\log 10(100) = 20$

b-) $\gg \ln(10^2/3) = \log(10^2/3) = 3.5066$

c-) $\gg \sin(7\pi/4) = \sin(7*pi/4) = -0.7071$

d-) $\gg \ln(1000+(2\pi)-2) = \log(1000+(2*pi)-2) = 6.9120$

e-) $\gg 5\sin(2.5^{3-\pi}) = 5*\sin(2.5^{(3-pi)}) = 3.8483$

f-) $\gg e^{\log 10} = \exp(\log(10)) = 10.000$

g-) $\gg 3\cos(\pi - 15) = 3*\cos(\pi-15) = 2.2791$

11-) $x = \frac{\sqrt{2}}{2}$ ve $y = \arccos(\pi/4)$ ise $\frac{x^y}{y^x}$ değeri nedir?

$\gg x = \text{sqrt}(2)/2; y = \text{acos}(pi/4);$

$\gg x^y/y^x$

ans =

1.0561

12-) $\sin^{-1}(\sin 30^\circ)$ değerini bulunuz?

$\gg \text{asin}(\sin(30)) > \text{asin}(\text{ind}(30))$

ans =

0.5236

13-) $x=3$ ise $y=x^6+\log(\pi)\sin x$ değerini bulunuz.

$\gg x=3;$

$\gg y = x^6 + \log(pi)*\sin(x)$

y =

729.1615

14-) Aşağıdaki faktöriyel hesaplarını yapınız.

Not : faktöriyel hesabında n pozitif bir tamsayı olmak üzere $n!=\text{factorial}(n)$, $n!=\text{prod}(1:n)$ veya $n!=\text{gamma}(n+1)$ ifadeleri kullanılabilir.

a-) $a!(5!-1)$

$\gg \text{factorial}(3)*(\text{factorial}(5)-1)$

ans =

714

$\gg \text{prod}(1:3)*(\text{prod}(1:5)-1)$

ans =

714

$\gg \text{gamma}(4)*(\text{gamma}(6)-1)$

ans =

714

b-)15!(9!-5!)

```
>> factorial(15)/(factorial(9)-factorial(5))
ans =
3.6048e+06
>> prod(1:15)/(prod(1:9)-prod(1:5))
ans =
3.6048e+06
```

15-) Bir FM radyo alıcısında $f= 50$ MHz , $w= 2\pi f$, $L=0.2$ mH (mili henry) , $C=0.25$ nF (nano Farad), $R= 0.33$ k Ω , $V_0=5$ mV ise V_R ifadesinin değerini bulunuz.

$$V_R = \frac{R}{\sqrt{R^2 + (wL - \frac{1}{wC})^2}} V_0 \quad \% \text{ hesap işlemlerinde birim uyuşmasına dikkat ediniz.}$$

```
>> f=50e6;
>> w=2*pi*f;
>> L=0.2e-3;
>> C=0.25e-9;
>> R=0.33e3;
>> V0=5e-3;
>> V_R=(R/sqrt(R^2+(w*L-1/(w*C))^2))*V0
V_R =
2.6266e-05
```

16-) Gürültü birimi olan desibel (dB) formülünde P_2 (ölçülen değer)=25 ve P_1 (referans seviyesi) = 10 ise dB değerini bulunuz.

```
dB=10*log10(P2/P1)
>> dB = 10*log10(25/10)
dB= 3.9794
```

17-) $V_{p2}=0.05$, $V_{p1}=0.01$, $R_p=15$, $R_s=45$ ve $K= 10^{-2}$ için ,

$$\frac{W_2}{W_1} = \frac{\frac{V_{p2}}{V_{p1}}}{\frac{R_p + R_s}{R_p + R_s} K} \text{ ifadesinin değerini bulunuz.}$$

```
>> vp2=0.05
vp2 =
0.0500
>> vp1=0.01
vp1 =
0.0100
>> rp=15
rp =
15
>> rs=45
rs =
45
>> k=10^-2
k =
0.0100
>> w1 = ((vp2)/(vp1))/ ((( rp + rs)/(rp - rs))/k)
w1 =
-0.0250
```

18-) $x = (0:0.1:2) * \pi$ vektörüne göre,

```
>> x=(0:0.1:2)*pi
```

x =

```
0 0.3142 0.6283 0.9425 1.2566 1.5708 1.8850 2.1991 2.5133 2.8274 3.1416
3.4558 3.7699 4.0841 4.3982 4.7124 5.0265 5.3407 5.6549 5.9690 6.2832
```

a-) ilk beş elemanı gösteriniz.

```
>> a=x(1:5)
```

a =

```
0 0.3142 0.6283 0.9425 1.2566
```

b-) 11'inci elemandan son elemana kadar elemanları gösteriniz.

```
>> b=x(11:end)
```

b =

```
3.1416 3.4558 3.7699 4.0841 4.3982 4.7124 5.0265 5.3407 5.6549 5.9690 6.2832
```

c-) Sırasıyla 9,8 ve 7'inci elemanları gösteriniz.

```
>> c=x(9:-1:7)
```

veya >> c=([9,8,7])

c =

```
2.5133 2.1991 1.8850
```

d-) Sondan 3 elemanını gösteriniz.

```
>> d=x(end:-1:end-2) veya d=x(end-2:end)
```

d =

```
6.2832 5.9690 5.6549
```

19-) $x = (0:\pi/2:2*\pi)$ vektörünü temel alarak ilk satırı x değerleri ,ikinci satırı $\sin x$ değerleri ve üçüncü satırı $\cos x$ değerlerinden oluşan bir matris elde ediniz.

```
>> x=(0:pi/2:2*pi)
```

x =

```
0 1.5708 3.1416 4.7124 6.2832
```

```
>> A=[x;sin(x);cos(x)]
```

A =

```
0 1.5708 3.1416 4.7124 6.2832
```

```
0 1.0000 0.0000 -1.0000 -0.0000
```

```
1.0000 0.0000 -1.0000 -0.0000 1.0000
```

20-) $v = [0:0.5:20]$ vektörü ile $A = [1 \ -2 \ 3 ; -3 \ 4 \ 5 ; 1 \ 0 \ 5]$ matrisleri verilmiştir.

```
>> v=[0:0.5:20], A=[1 -2 3;-3 4 5;1 0 5]
```

v =

```
0 0.5000 1.0000 1.5000 2.0000 2.5000 3.0000 3.5000 4.0000 4.5000 5.0000 5.5000 6.0000
6.5000 7.0000 7.5000 8.0000 8.5000 9.0000 9.5000 10.0000 10.5000 11.0000 11.5000 12.0000
12.5000 13.0000 13.5000 14.0000 14.5000 15.0000 15.5000 16.0000 16.5000 17.0000 17.5000
18.0000 18.5000 19.0000 19.5000 20.0000
```

A =

```
1 -2 3
```

```
-3 4 5
```

```
1 0 5
```

a-) v vektörünün ilk üç elemanı ile A matrisinin ilk sütununu çarpınız ve sonucu yorumlayınız.

```
>> v(1:3)*A(:,1)
```

ans =

```
-0.5000
```

b-)

```
>> A(3,:).*v(1:3)
```

ans =

```
0 0 5
```

21-)

```
>> for i=1:4
a(i,1)=1;
a(1,i)=1;
a(i,i)=1;
end
>> a
a =
1  1  1  1
1  1  0  0
1  0  1  0
1  0  0  1
```

22-)

```
>> for i=1:6
x(i) = (i-1)*0.1;
end
>> x
x =
0  0.1000  0.2000  0.3000  0.4000  0.5000
```

23-)

```
>> a=[1;5];
>> x=[5;1];
>> for i=1:3
x=a.*3;
end
>> x
x =
3
15
```

24-)

```
>> t=1 ; n=0;
>> while t<10
t=t*3;
n=n+1;
deger(n)=t;
end
>> deger'
ans =
3
9
27
```

25-)

```
>> a=10;
>> while a>2
a=a/2;
sonuc=a;
a=a-2;
if a<1
break
end
end
>> a
a =
-0.5000
>> sonuc
sonuc =
1.5000
```

26-)

```
>> sayac=1; num=2;
>> while sayac < 20
num=num+5;
sayac=sayac+2;
if sayac>=10
break
end
end
>> num
num =
27
>> sayac
sayac =
11
```

27-)

$$A = \frac{b_1 + b_2}{2} \cdot h$$

```
>> b1= 0.3;b2=1.5;h=2;
>> A= (b1+b2)*h/2
A =
1.8000
```

28 -) Girilen bir sayının 10 luk tabanda ve doğal logaritmasını bulan ve görüntüleyen bir program yazınız.

```
a=input('Bir sayı giriniz');
b=log10(a);
c=log(a);
fprintf('\n10 luk tabanda=%f',b)
fprintf('\ndoğal logaritma =%f', c)
>> deneme12
Bir sayı giriniz5
10 luk tabanda=0.698970
doğal logaritma =1.609438
```

29-) Derece olarak girilen açının sinüs ve cosinüs değerini bulan bir program yazınız.
% Bu program derece olarak %girilen açının sinüs ve %cosinüs değerlerini bulur

```
aci=input('Bir aci degeri giriniz');  
aci=aci*pi/180;  
d1=sin(aci);  
d2=cos(aci);  
fprintf('\n sinus degeri=%f',d1)  
fprintf('\n cosinus degeri= %f',d2)  
>> deneme12  
Bir aci degeri giriniz45  
sinus degeri=0.707107  
cosinus degeri= 0.707107
```

30-)

```
>> dz1=1:3:10  
dz1 =  
1 4 7 10  
>> dz2=11:2:20  
dz2 =  
11 13 15 17 19  
>> dz3=[dz1,dz2]  
dz3 =  
1 4 7 10 11 13 15 17 19
```

31-)

```
>> z=[1-i, 2+3i, 4-9i]  
z =  
1.0000 - 1.0000i 2.0000 + 3.0000i 4.0000 - 9.0000i  
>> z2=conj(z)  
z2 =  
1.0000 + 1.0000i 2.0000 - 3.0000i 4.0000 + 9.0000i  
>> r1=real(z)  
r1 =  
1 2 4  
>> s1=imag(z)  
s1 =  
-1 3 -9
```

32-)

!!!!!!(hatalı)

Seri RC devresinde Vc geriliminin 0-10ms arasında değişimini çizdiriniz. (R=100;C=50 μ F,Vk=5V)

```
Vc=5*(1-e^(t/R*C))  
R=30;  
C=50E-6;  
Vk=5;  
T=R*C;  
t=0:10e-6:10e-3;  
Vc=5*(1-exp(-t./T));  
plot(t,Vc,'r')  
title('kapasitör geriliminin değişimi')  
xlabel('zaman')  
ylabel('gerilim')  
grid on
```


33-) Şekil 1’de görülen devrede $R=10\Omega$, $C=0.1F$ ve $V_g=5V$ dur. $t=0$ anında S anahtarı kapanıyor. $t=2$. sn de açılıp $t=3$. sn de tekrar kapanıyor. Kapasitör geriliminin 0-6 sn arasındaki değişimini elde ediniz. ($V_c(0)=0$)

Anahtar kapalı

$$\frac{dV_c}{dt} = \frac{1}{R \cdot C} \cdot (V_g - V_c)$$

Anahtar açık

$$\frac{dV_c}{dt} = 0$$

```
function dx=rc1(t,x)
Vg=5; R=10; C=0.1;
if t<2 | t>3
    dVc = 1/(R*C) * (Vg - Vc);
    dx(1)=(Vg-x(1))/(R*C);
else
    dVc = 0;
    dx(1)=0;
end
```

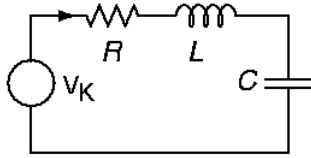
34-) Seri bir RLC devresi sinüzoidal bir gerilim kaynağından beslenmektedir. Kullanıcının girdiği R , L , C , w ve kaynağın genlik değerlerine bağlı olarak kaynaktan çekilen akımı, devrenin empedansını ve rezonans frekansını hesaplayan bir m.function oluşturunuz. $X_L=X_C$

```
fr = sqrt(1/(2*pi*L*C))
function [Z,fr]=rlc(R,L,C,w)
XC=1/(j*w*C);
XL=j*w*L;
Z=R+XL+XC;
fr=1/sqrt(2*pi*L*C)
```

35-) Seri bir RLC devresi sinüzoidal bir gerilim kaynağından beslenmektedir. Kullanıcının girdiği R , L , C , w ve kaynağın genlik değerlerine bağlı olarak kaynaktan çekilen akımı, devrenin empedansını ve rezonans frekansını hesaplayan bir m.function oluşturunuz.

```
function [I,Z,fr]=soru(R, L, C, w,A)
XL=j*w*L;XC=1/(j*w*c);
Z= (R+XL+XC);
I=A/Z; fr=1/(2*pi*sqrt(L*C));
```

36-) Aşağıdaki devrede $R=20\Omega$, $C=0.01F$, $L=0.5H$ ve $V_K=20\sin(10t)$ olarak verilmiştir.



Devredeki akımı ve elemanların gerilimlerini kompleks olarak bulacak ve çizim ekranını dörde bölerek, sırasıyla devreden geçen akımın, V_R , V_L , V_C nin zamana göre değişimini 0-50 ms aralığında çizdirecek bir **m-dosya** oluşturunuz.

```
R=20 ; C=0.01; L=0.5;
VK=20;w=10;
XL=j*w*L;
XC=1/(j*w*C);
Z=R+XL+XC;
I=VK/Z;
VR=R*I;
VC=I*XC;
VL=I*XL;
t=linspace(0,1,500);
subplot(411)
plot(t,abs(I)*sin(w*t+angle(I)))
subplot(412)
plot(t,abs(VR)*sin(w*t+angle(VR)))
subplot(413)
plot(t,abs(VL)*sin(w*t+angle(VL)))
subplot(414)
plot(t,abs(VC)*sin(w*t+angle(VC)))
```